



Course Title: Cloud Computing W26

Course Code: CISC 886

Instructor: Dr. Anwar Hossain

Term: Winter 2026

Ubuntu Assistant: Fine-Tuned LLM for Technical Issue Resolution Readme

Group: 12

Name:

ID:

Email:

Farida El Ghabary

20595451

25nplx@queensu.ca

Rodina Khallaf

20296324

25cfsf@queensu.ca

Salma Elsherif

20596332

25bwpr@queensu.ca

2025-2026

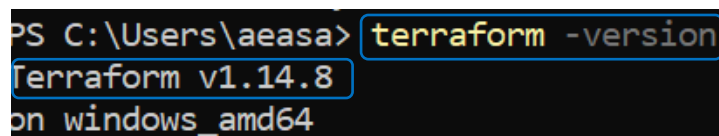
GitHub Repository Requirements

Steps to Replicate the project:

Terraform was installed using the Chocolatey package manager. This process began by executing a PowerShell command to install Chocolatey, which involved bypassing execution policy restrictions and enabling secure TLS communication:

- Set-ExecutionPolicy Bypass -Scope Process -Force;
- [System.Net.ServicePointManager]::SecurityProtocol = [System.Net.ServicePointManager]::SecurityProtocol -bor 3072;
- iex ((New-Object System.Net.WebClient).DownloadString('https://community.chocolatey.org/install.ps1'))

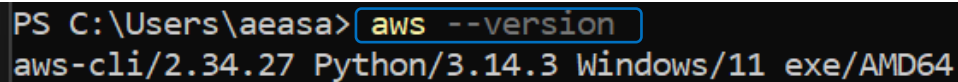
Once Chocolatey was successfully installed, Terraform was installed using the command `choco install terraform -y`. The installation was then verified by executing `terraform -version`, which confirmed that the tool was correctly installed and accessible in the system environment, as illustrated in Fig. (1).



```
PS C:\Users\aeasa> terraform -version
Terraform v1.14.8
on windows_amd64
```

Fig. (1): verification of Terraform using the Chocolatey package manager in a Windows PowerShell environment.

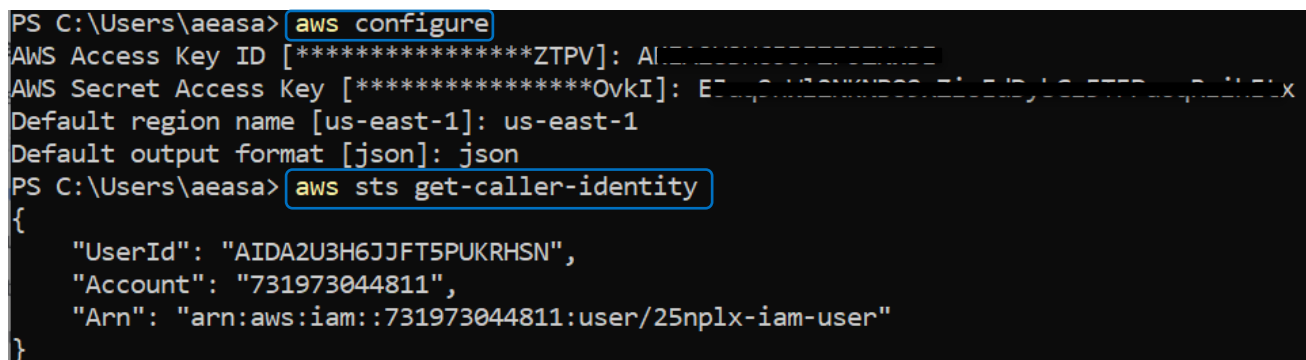
The AWS CLI was installed by first downloading and installing the official package. After completing the installation, the terminal was restarted to ensure the environment variables were properly updated. The installation was then verified by executing the command `aws --version`, as shown in Fig. (2).



```
PS C:\Users\aeasa> aws --version
aws-cli/2.34.27 Python/3.14.3 Windows/11 exe/AMD64
```

Fig. (2): Verification of AWS CLI installation using the `aws --version` command in the terminal.

An IAM user was created with the AdministratorAccess policy attached. An access key pair was then generated for the user and configured locally using the `aws configure` command. The configuration was validated by executing `aws sts get-caller-identity`, confirming successful authentication and correct identity association, as shown in Fig. (3).



```
PS C:\Users\aeasa> aws configure
AWS Access Key ID [*****ZTPV]: AIDA2U3H6JJFT5PUKRHSN
AWS Secret Access Key [*****OvkI]: ETqP4Mf2Kw1230-21-4IDJG-ITTP-44-24-11-1x
Default region name [us-east-1]: us-east-1
Default output format [json]: json
PS C:\Users\aeasa> aws sts get-caller-identity
{
  "UserId": "AIDA2U3H6JJFT5PUKRHSN",
  "Account": "731973044811",
  "Arn": "arn:aws:iam::731973044811:user/25nplx-iam-user"
}
```

Fig. (3): Verification of AWS CLI configuration using IAM user credentials via the STS GetCallerIdentity command.

The Ubuntu Dialogue Corpus, obtained from the McGill University dataset repository in tab-separated values (TSV) format (download from this link <https://dataset.cs.mcgill.ca/ubuntu-corpus-1.0/?>), was converted to the Apache Parquet columnar storage format to facilitate efficient distributed processing on Amazon EMR. This conversion was performed by executing the `convert_to_parquet.py` script locally. As depicted in Figure 4, the

terminal output logs the conversion progress at regular intervals, reporting cumulative file counts and corresponding line totals. The script processed approximately 1.85 million source files encompassing over 16.5 million individual rows, completing the conversion in approximately 140 minutes. The resulting Parquet file, ubuntu_raw.parquet, occupies 735.5 MB on disk - a substantially more compact and query-efficient representation compared to the original TSV source - and constitutes the primary input for the subsequent distributed preprocessing stage.

```
Microsoft Windows [Version 10.0.26100.7623]
(c) Microsoft Corporation. All rights reserved.

C:\Users\DPQUAI250118>doskey /history

C:\Users\DPQUAI250118>cd downloads

C:\Users\DPQUAI250118>Downloads>python convert_to_parquet
C:\Users\DPQUAI250118>AppData\Local\Python\Python39-64\python.exe: can't open file 'C:\Users\DPQUAI250118\Downloads\convert_to_parquet': [Errno 2] No such file or directory

C:\Users\DPQUAI250118>Downloads>python convert_to_parquet.py
Starting conversion...
Files: 10,000 | Lines: 100,000 | Minutes: 0.58
Files: 20,000 | Lines: 200,000 | Minutes: 1.18
Files: 30,000 | Lines: 300,000 | Minutes: 1.79
Files: 40,000 | Lines: 400,000 | Minutes: 2.39
Files: 50,000 | Lines: 500,000 | Minutes: 2.99
Files: 60,000 | Lines: 600,000 | Minutes: 3.72
Files: 70,000 | Lines: 700,000 | Minutes: 4.42
Files: 80,000 | Lines: 876,430 | Minutes: 5.16
Files: 90,000 | Lines: 966,430 | Minutes: 5.86
Files: 100,000 | Lines: 1,096,430 | Minutes: 6.59
Files: 110,000 | Lines: 1,206,430 | Minutes: 7.36
Files: 120,000 | Lines: 1,316,430 | Minutes: 8.09
Files: 130,000 | Lines: 1,426,430 | Minutes: 8.81
Files: 140,000 | Lines: 1,536,430 | Minutes: 9.52
Files: 150,000 | Lines: 1,646,430 | Minutes: 10.24
Files: 160,000 | Lines: 1,756,430 | Minutes: 10.92
Files: 170,000 | Lines: 1,866,430 | Minutes: 11.62
Files: 180,000 | Lines: 2,036,329 | Minutes: 12.38
Files: 190,000 | Lines: 2,156,329 | Minutes: 13.1
Files: 200,000 | Lines: 2,276,329 | Minutes: 13.82
Files: 210,000 | Lines: 2,396,329 | Minutes: 14.51
Files: 220,000 | Lines: 2,516,329 | Minutes: 15.23
Files: 230,000 | Lines: 2,679,453 | Minutes: 15.95
Files: 240,000 | Lines: 2,809,453 | Minutes: 16.68
Files: 250,000 | Lines: 2,939,453 | Minutes: 17.37
Files: 260,000 | Lines: 3,069,453 | Minutes: 18.07
Files: 270,000 | Lines: 3,199,453 | Minutes: 18.4
Files: 280,000 | Lines: 3,379,457 | Minutes: 19.51
Files: 290,000 | Lines: 3,519,457 | Minutes: 20.26
Files: 300,000 | Lines: 3,659,457 | Minutes: 21.01
Files: 310,000 | Lines: 3,828,478 | Minutes: 21.79
Files: 320,000 | Lines: 3,978,478 | Minutes: 22.48
Files: 330,000 | Lines: 4,120,478 | Minutes: 23.37
Files: 340,000 | Lines: 4,260,634 | Minutes: 24.15
Files: 350,000 | Lines: 4,400,634 | Minutes: 24.91
Files: 360,000 | Lines: 4,649,886 | Minutes: 25.74
Files: 370,000 | Lines: 4,819,886 | Minutes: 26.53
Files: 380,000 | Lines: 5,011,192 | Minutes: 27.36
Files: 390,000 | Lines: 5,191,192 | Minutes: 28.21
Files: 400,000 | Lines: 5,391,549 | Minutes: 29.08
Files: 410,000 | Lines: 5,595,796 | Minutes: 29.76
Files: 420,000 | Lines: 5,800,965 | Minutes: 30.53
Files: 430,000 | Lines: 6,015,965 | Minutes: 31.33
Files: 440,000 | Lines: 6,241,055 | Minutes: 32.13
Files: 450,000 | Lines: 6,480,951 | Minutes: 32.99
Files: 460,000 | Lines: 6,737,679 | Minutes: 33.77
Files: 470,000 | Lines: 7,003,975 | Minutes: 34.47
Files: 480,000 | Lines: 7,284,603 | Minutes: 35.15
Files: 490,000 | Lines: 7,616,580 | Minutes: 35.88
Files: 500,000 | Lines: 7,960,350 | Minutes: 36.62
Files: 510,000 | Lines: 7,476,580 | Minutes: 37.33
Files: 520,000 | Lines: 7,566,580 | Minutes: 38.05
Files: 530,000 | Lines: 7,536,580 | Minutes: 38.85
Files: 540,000 | Lines: 7,566,580 | Minutes: 39.6
Files: 550,000 | Lines: 7,596,580 | Minutes: 40.45
Files: 560,000 | Lines: 7,626,580 | Minutes: 41.27
Files: 570,000 | Lines: 7,656,580 | Minutes: 42.04

Files: 1,450,000 | Lines: 12,991,585 | Minutes: 100.94
Files: 1,460,000 | Lines: 13,051,585 | Minutes: 100.75
Files: 1,470,000 | Lines: 13,111,585 | Minutes: 110.57
Files: 1,480,000 | Lines: 13,171,585 | Minutes: 111.32
Files: 1,490,000 | Lines: 13,231,585 | Minutes: 112.06
Files: 1,500,000 | Lines: 13,291,585 | Minutes: 112.81
Files: 1,510,000 | Lines: 13,351,585 | Minutes: 113.64
Files: 1,520,000 | Lines: 13,411,585 | Minutes: 114.41
Files: 1,530,000 | Lines: 13,696,497 | Minutes: 115.25
Files: 1,540,000 | Lines: 13,766,497 | Minutes: 115.97
Files: 1,550,000 | Lines: 13,836,497 | Minutes: 116.69
Files: 1,560,000 | Lines: 13,906,497 | Minutes: 117.41
Files: 1,570,000 | Lines: 13,976,497 | Minutes: 118.15
Files: 1,580,000 | Lines: 14,046,497 | Minutes: 118.91
Files: 1,590,000 | Lines: 14,116,497 | Minutes: 119.63
Files: 1,600,000 | Lines: 14,186,497 | Minutes: 120.36
Files: 1,610,000 | Lines: 14,256,497 | Minutes: 121.09
Files: 1,620,000 | Lines: 14,326,497 | Minutes: 121.83
Files: 1,630,000 | Lines: 14,396,497 | Minutes: 122.54
Files: 1,640,000 | Lines: 14,466,497 | Minutes: 123.26
Files: 1,650,000 | Lines: 14,536,497 | Minutes: 123.97
Files: 1,660,000 | Lines: 14,758,170 | Minutes: 124.77
Files: 1,670,000 | Lines: 14,838,170 | Minutes: 125.52
Files: 1,680,000 | Lines: 14,918,170 | Minutes: 126.25
Files: 1,690,000 | Lines: 14,998,170 | Minutes: 126.97
Files: 1,700,000 | Lines: 15,078,170 | Minutes: 127.77
Files: 1,710,000 | Lines: 15,158,170 | Minutes: 128.53
Files: 1,720,000 | Lines: 15,238,170 | Minutes: 129.28
Files: 1,730,000 | Lines: 15,318,170 | Minutes: 129.98
Files: 1,740,000 | Lines: 15,398,170 | Minutes: 130.76
Files: 1,750,000 | Lines: 15,478,170 | Minutes: 131.52
Files: 1,760,000 | Lines: 15,558,170 | Minutes: 132.31
Files: 1,770,000 | Lines: 15,783,186 | Minutes: 133.16
Files: 1,780,000 | Lines: 15,843,186 | Minutes: 133.94
Files: 1,790,000 | Lines: 15,933,186 | Minutes: 134.75
Files: 1,800,000 | Lines: 16,023,186 | Minutes: 135.53
Files: 1,810,000 | Lines: 16,113,186 | Minutes: 136.31
Files: 1,820,000 | Lines: 16,203,186 | Minutes: 137.08
Files: 1,830,000 | Lines: 16,293,186 | Minutes: 137.88
Files: 1,840,000 | Lines: 16,383,186 | Minutes: 138.62
Files: 1,850,000 | Lines: 16,473,186 | Minutes: 139.39

Converting to DataFrame...
Total rows: 16,587,830
Columns: ['dialogue_id', 'timestamp', 'speaker', 'addressee', 'message']
Sample:
dialogue_id    timestamp speaker addressee    message
0      10/1.tsv  2005-05-26T16:51:00.000Z    we2by      hi
1      10/1.tsv  2005-05-26T16:51:00.000Z    we2by      sudo echo Y > /sys/module/usbcore/parameters/o...
2      10/1.tsv  2005-05-26T16:51:00.000Z    we2by      why do I get permission denied????

Saving to Parquet...
✓ Done in 100.24 minutes
✓ Saved to: C:\Users\DPQUAI250118\Downloads\ubuntu_raw.parquet
✓ File size: 0.72 GB
```

Fig. (4): Terminal output of the convert_to_parquet.py script, illustrating the incremental processing of the Ubuntu Dialogue Corpus from TSV format to Apache Parquet. The log displays cumulative file and line counts at intervals of 10,000 files, indicating sustained throughput over approximately 140 minutes.

Following the successful local conversion of the Ubuntu Dialogue Corpus to Parquet format, the resulting file was uploaded to the designated Amazon S3 bucket, 25nplx-cisc886-bucket, which was provisioned in the us-east-1 region using the AWS CLI command `aws s3 mb s3://25nplx-cisc886-bucket --region us-east-1`. The file transfer was performed via the S3 console interface. As shown in Figure 5, the upload status panel confirms that the single file `ubuntu_raw.parquet`, totalling 735.5 MB, was transferred successfully, with zero failed objects recorded. The file is stored using the S3 Standard storage class, making it durably available for retrieval by the EMR cluster during the distributed preprocessing phase. The S3 bucket thereby serves as the centralised data repository for all intermediate and final pipeline artefacts.

```
PS C:\Users\aeasa> aws s3 mb s3://25nplx-cisc886-bucket --region us-east-1
make_bucket: 25nplx-cisc886-bucket
PS C:\Users\aeasa> aws s3 ls
2026-04-29 10:09:05 25nplx-cisc886-bucket
```

Fig. (5): Create the S3 25nplx-cisc886-bucket

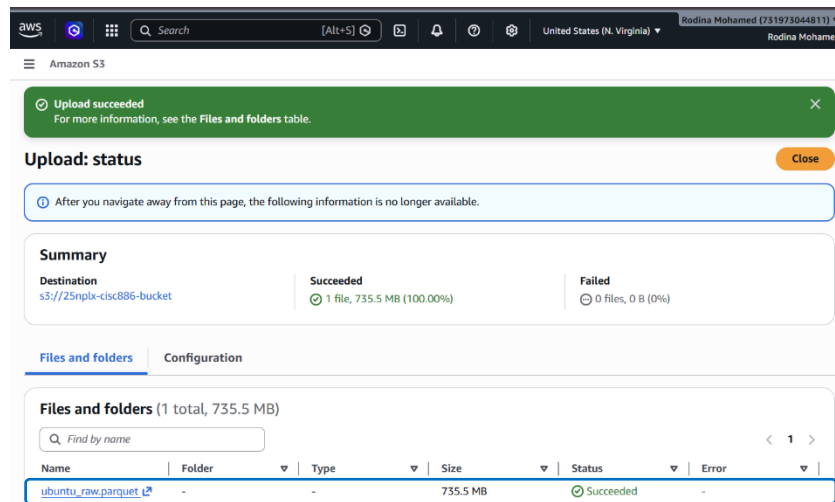


Fig. (6): Amazon S3 console view confirming successful upload of the `ubuntu_raw.parquet` file (735.5 MB) to the S3 bucket 25nplx-cisc886-bucket. The upload status banner indicates that the transfer completed without errors.

The cloud infrastructure for this project was defined entirely as code using Terraform, organised into three logically separated modules to enable independent provisioning and teardown of each architectural layer. As illustrated in Fig. (7), the initial directory structure comprises the `foundation/` module, which encapsulates the Virtual Private Cloud (VPC) definition along with public and private subnets, an Internet Gateway, a NAT Gateway, an S3 VPC endpoint, and the associated IAM roles for both EMR and EC2 services. The `emr/` module contains the cluster definition for the Apache Spark preprocessing job, while the `deployment/` module defines the Bastion host and the LLM inference EC2 instance. This modular organisation ensures a clean separation of concerns and facilitates reproducible, incremental infrastructure management throughout the project lifecycle.

```
Terraform_files/
├── foundation/
│   ├── main.tf      ← VPC, subnets, IGW, NAT, S3 endpoint
│   └── iam.tf       ← IAM roles for EMR + EC2
├── emr/
│   └── emr.tf       ← EMR cluster
└── deployment/
    └── deployment.tf ← Bastion + LLM EC2
```

Fig. (7): Initial Terraform project directory structure, comprising three modules: `foundation/` (VPC, subnets, Internet Gateway, NAT Gateway, and S3 endpoint), `emr/` (EMR cluster definition), and `deployment/` (Bastion host and LLM EC2 instance).

Secure shell (SSH) access to the EMR cluster and EC2 instances requires a cryptographic key pair. A 4096-bit RSA key pair was generated using the `ssh-keygen` utility, with the output directed to the `utilities/` subdirectory of the Terraform project. The command was executed without a passphrase to support automated infrastructure workflows. As shown in Fig. (8), the terminal confirms the successful creation of both the private key file (`25nplx-keypair`) and the corresponding public key file (`25nplx-keypair.pub`), and displays the SHA256 fingerprint alongside the randomart visual representation of the key. The public key was subsequently imported into AWS using the `aws ec2 import-key-pair` command, making it available for association with EC2 and EMR instances provisioned by Terraform.

Generate the Key Pair using `ssh-keygen -t rsa -b 4096 -f "your_terraform_\Terraform_files\25nplx-keypair"`. When it asks for passphrase - press Enter twice to skip. This would make the keys that would be use.

```
PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities> ssh-keygen -t rsa -b 4096 -f C:\Users\aeasa\.ssh\25nplx-keypair
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\aeasa\.ssh\25nplx-keypair
Your public key has been saved in C:\Users\aeasa\.ssh\25nplx-keypair.pub
The key fingerprint is:
SHA256:mmwMwMaSDqZsTI/oe4C+jnQoQkC1c6MG0I/aABKO+MQ aeasa@LAPTOP-QEA28M15
The key's randomart image is:
+---[RSA 4096]---+
|o+..
|O= .
|X=E o
|%=.= .
|+@.+ S
|*+ + o
|++.. *
|=.O. .
|O+O
+---[SHA256]---+
```

Fig. (8): Terminal output of the `ssh-keygen` command generating a 4096-bit RSA key pair (`25nplx-keypair`) in the Windows PowerShell environment. The output displays the key fingerprint and randomart image confirming successful generation of both the private and public key files.

Following the generation of the SSH key pair and the preparation of the PySpark preprocessing script, the Terraform project directory was augmented with two additional components. As illustrated in Fig. (9), the `utilities/` directory was populated with the generated `25nplx-keypair` private key and `25nplx-keypair.pub` public key files, while the `preprocessing/` directory was populated with the EMR bootstrap shell script (`bootstrap.sh`) - responsible for installing required Python dependencies on cluster nodes - and the PySpark data preprocessing script (`ubuntu_preprocessing.py`). This constitutes the complete file structure required to provision the EMR cluster and execute the distributed preprocessing pipeline. Both preprocessing files were subsequently uploaded to the S3 scripts/ prefix to make them accessible to the EMR cluster at runtime.

```
Terraform_files/
├── foundation/
│   ├── main.tf    + VPC, subnets, IGW, NAT, S3 endpoint
│   └── iam.tf      + IAM roles for EMR + EC2
├── emr/
│   └── emr.tf      + EMR cluster
├── deployment/
│   └── deployment.tf + Bastion + LLM EC2
├── utilities/
│   ├── 25nplx-keypair
│   └── 25nplx-keypair.pub
└── preprocessing
    ├── bootstrap.sh
    └── ubuntu_preprocessing.py
```

Fig. (9): Updated Terraform project directory structure following the addition of the `utilities/` directory (containing the generated RSA key pair) and the `preprocessing/` directory (containing `bootstrap.sh` and `ubuntu_preprocessing.py`), completing the full project file layout required for EMR provisioning.

Provisioning of the foundational networking and identity infrastructure was initiated by executing the standard Terraform workflow within the `foundation/` module directory. The `terraform init` command downloaded and installed the required AWS provider plugin (`hashicorp/aws v5.100.0`) and successfully initialised the backend. Execution of `terraform validate` confirmed that the configuration files were syntactically and semantically valid. The subsequent

terraform plan command produced an execution plan enumerating all resources scheduled for creation, including Elastic IP allocation, VPC configuration, public and private subnets, an Internet Gateway, a NAT Gateway, route tables, and the associated S3 VPC endpoint. As shown in Fig. (10), all planned resources are designated for creation (indicated by +), with the majority of attribute values reported as "known after apply", reflecting appropriate dependency resolution by Terraform's execution engine.

```
PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\foundation> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\foundation> terraform validate
Success! The configuration is valid.

PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\foundation> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_eip.nat will be created
+ resource "aws_eip" "nat" {
+   allocation_id    = (known after apply)
+   arn              = (known after apply)
+   association_id   = (known after apply)
+   carrier_ip       = (known after apply)
+   customer_owned_ip = (known after apply)
+   domain           = "vpc"
+   id               = (known after apply)
}
```

Fig. (10): Terminal output of the sequential execution of terraform init, terraform validate, and terraform plan commands within the foundation/ module directory. The output confirms successful provider initialisation (hashicorp/aws v5.100.0), configuration validity, and the planned creation of networking resources including Elastic IP, VPC, subnets, Internet Gateway, NAT Gateway, and route tables.

Upon confirmation of the execution plan, terraform apply was executed to provision the foundation infrastructure. As illustrated in Fig. (11), Terraform successfully created all 19 planned resources without error. The outputs section of the apply result exposes the key resource identifiers required by downstream modules: the vpc_id (vpc-0962a0563e0d101bb), private_subnet_id (subnet-099bb09528fbee1b8), public_subnet_id (subnet-06ab8b608fd0becd0), availability_zone (us-east-1a), ec2_deployment_profile_name, and the emr_service_role_arn. These output values are consumed programmatically by the emr/ and deployment/ modules via Terraform data sources, enabling the modular architecture to reference shared infrastructure without duplicating resource definitions.

```
PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\foundation> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_eip.nat will be created
+ resource "aws_eip" "nat" {
+   allocation_id    = (known after apply)
+   arn              = (known after apply)
+   association_id   = (known after apply)
+   carrier_ip       = (known after apply)
+   customer_owned_ip = (known after apply)
+   domain           = "vpc"
+   id               = (known after apply)
+   instance         = (known after apply)
+   ipam_pool_id     = (known after apply)
+   network_border_group = (known after apply)
+   network_interface = (known after apply)
+   private_dns       = (known after apply)
+   private_ip        = (known after apply)
+   ptr_record        = (known after apply)
+   public_dns        = (known after apply)
+   public_ip         = (known after apply)
+   public_ipv4_pool   = (known after apply)
+   tags_all          = (known after apply)
+   vpc               = (known after apply)
}
```



```

Apply complete! Resources: 19 added, 0 changed, 0 destroyed.

Outputs:
availability_zone = "us-east-1a"
ec2_deployment_profile_name = "25nplx-ec2-deployment-profile"
emr_ec2_profile_name = "25nplx-emr-ec2-profile"
emr_service_role_arn = "arn:aws:iam::731973844811:role/25nplx-emr-service-role"
private_subnet_id = "subnet-099bb09528fbee1b8"
public_subnet_id = "subnet-06ab8b68f0b0ecd0"
vpc_id = "vpc-0962a0563e0d101bb"

```

Fig. (11): Terminal output of terraform apply for the foundation/ module, confirming the successful creation of 19 AWS resources. The outputs section displays the provisioned resource identifiers, including VPC ID, subnet IDs, availability zone, and IAM profile names, confirming complete foundation infrastructure deployment.

To associate the locally generated SSH key pair with AWS EC2 resources, the public key component was imported into the AWS EC2 key pair registry using the `aws ec2 import-key-pair` command (`aws ec2 import-key-pair --key-name "25nplx-keypair" --public-key-material fileb://25nplx-keypair.pub`), referencing the public key file via the `fileb://` URI scheme. As shown in Fig. (12), the command returns a JSON object confirming the successful registration of the key pair under the name `25nplx-keypair`, along with its assigned `KeyPairId` (`key-07c36d233d39f8948`) and SHA256 fingerprint. Subsequent execution of `aws ec2 describe-key-pairs --key-names 25nplx-keypair` further validates the import by returning the full key pair metadata, including the creation timestamp and tag information. The registered key pair is subsequently referenced in the EMR and EC2 Terraform resource definitions to enable SSH access to provisioned instances.

```

PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities> aws ec2 import-key-pair --key-name "25nplx-keypair" --public-key-material fileb://25nplx-keypair.pub
{
  "KeyFingerprint": "df:6f:49:c1:69:46:ff:fe:a6:66:9e:37:bd:fe:3b:93",
  "KeyName": "25nplx-keypair",
  "KeyPairId": "key-07c36d233d39f8948"
}

PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities> aws ec2 describe-key-pairs --key-names 25nplx-keypair
{
  "KeyPairs": [
    {
      "KeyPairId": "key-07c36d233d39f8948",
      "KeyType": "rsa",
      "Tags": [],
      "CreateTime": "2026-04-30T11:07:15.959000+00:00",
      "KeyName": "25nplx-keypair",
      "KeyFingerprint": "df:6f:49:c1:69:46:ff:fe:a6:66:9e:37:bd:fe:3b:93"
    }
  ]
}

```

Fig. (12): Terminal output confirming the successful importation of the RSA public key (`25nplx-keypair`) into AWS EC2 via the `aws ec2 import-key-pair` command, along with subsequent verification using `aws ec2 describe-key-pairs`, which returns the associated key ID, fingerprint, and creation timestamp.

Prior to EMR cluster provisioning, the EMR bootstrap script and PySpark preprocessing application were uploaded to the designated S3 scripts/ prefix to ensure their availability to the cluster at runtime. The `bootstrap.sh` file - which installs required Python libraries on all cluster nodes during the bootstrapping phase - and `ubuntu_preprocessing.py` - the PySpark application responsible for dialogue grouping, quality scoring, and train/validation/test partitioning - were transferred to S3 using the `aws s3 cp` command (`aws s3 cp bootstrap.sh s3://25nplx-cisc886-bucket/scripts/bootstrap.sh`, and `aws s3 cp ubuntu_preprocessing.py s3://25nplx-cisc886-bucket/scripts/ubuntu_preprocessing.py` commands.). As illustrated in Fig. (13), the terminal output confirms the successful upload of both files, and the S3 console view shows that the bucket `25nplx-cisc886-bucket` now contains three objects: the `scripts/` folder (with `bootstrap.sh` at 450 B and `ubuntu_preprocessing.py` at 18.4 KB), and the previously uploaded `ubuntu_raw.parquet` dataset file (735.5 MB, last modified 30 April 2026).

```

PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\preprocessing> aws s3 cp bootstrap.sh s3://25nplx-cisc886-bucket/scripts/bootstrap.sh
>> aws s3 cp ubuntu_preprocessing.py s3://25nplx-cisc886-bucket/scripts/ubuntu_preprocessing.py
upload: .\bootstrap.sh to s3://25nplx-cisc886-bucket/scripts/bootstrap.sh
upload: .\ubuntu_preprocessing.py to s3://25nplx-cisc886-bucket/scripts/ubuntu_preprocessing.py

```

Fig. (13): Terminal output confirming the successful upload of `bootstrap.sh` and `ubuntu_preprocessing.py` to the `scripts/` prefix of the S3 bucket `25nplx-cisc886-bucket` via the `aws s3 cp` command. The S3 console view shows both files present in the `scripts/` folder alongside the previously uploaded `ubuntu_raw.parquet` dataset file.

Fig. (14) provides a detailed S3 console view of the scripts/ prefix within the 25nplx-cisc886-bucket bucket, confirming the presence of both required EMR runtime files. The bootstrap.sh shell script (450 bytes) is referenced in the EMR Terraform configuration as a bootstrap action, ensuring that any additional Python packages required by the preprocessing application are installed on cluster nodes before the PySpark step is submitted. The ubuntu_preprocessing.py file (18.4 KB) constitutes the main PySpark application that is submitted as the EMR step. Both files were uploaded on 30 April 2026 and are stored under the S3 Standard storage class, ensuring high durability and immediate availability to the EMR cluster upon launch.

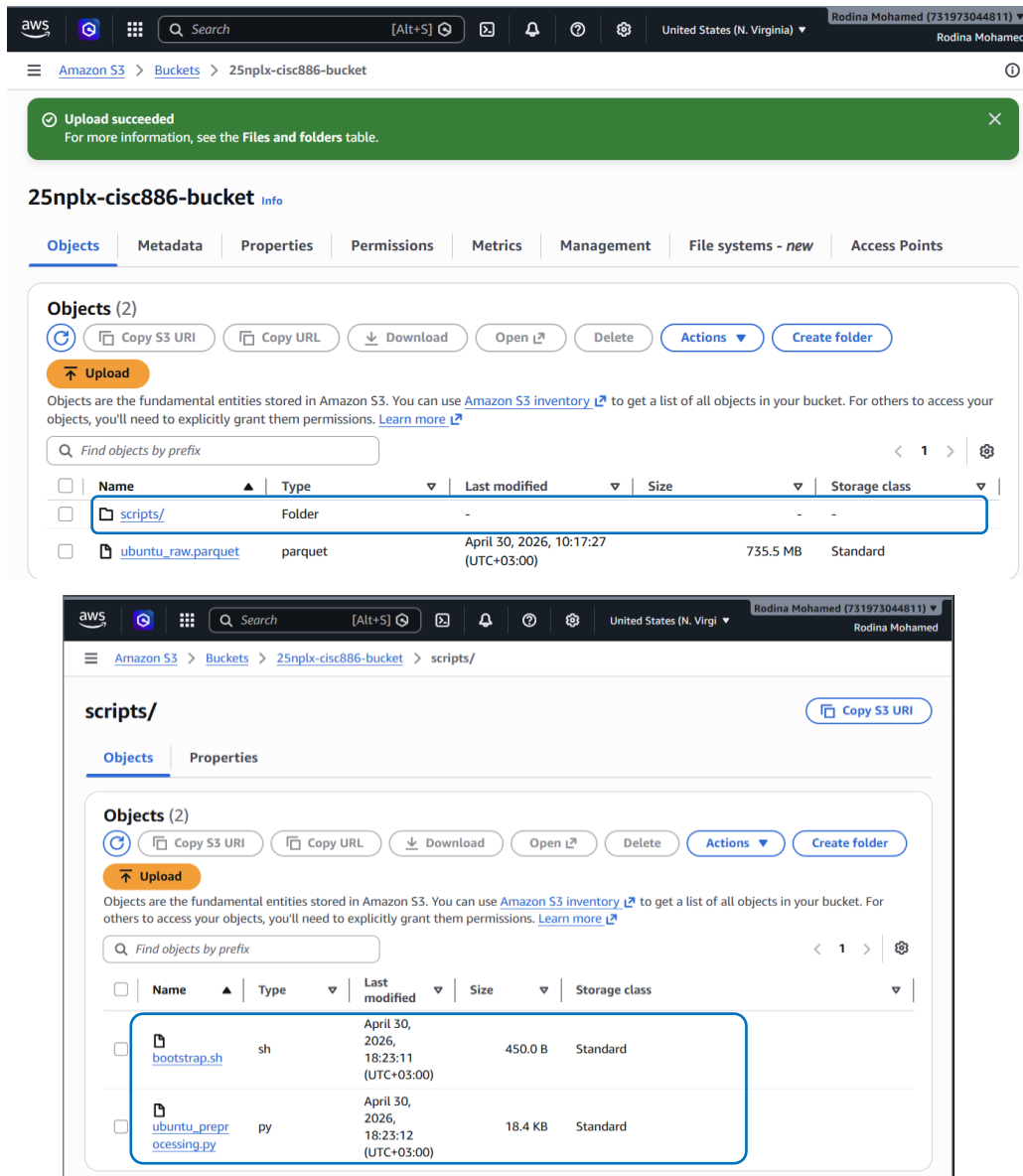


Fig. (14): S3 console view of the scripts/ prefix within 25nplx-cisc886-bucket, showing the two uploaded files: bootstrap.sh (450 B) and ubuntu_preprocessing.py (18.4 KB), both stored in the S3 Standard storage class and uploaded on 30 April 2026.

With the foundation infrastructure operational and all prerequisite assets uploaded to S3, the EMR module was initialised and planned using the Terraform workflow. As shown in Fig. (15), terraform init successfully installed the AWS provider plugin within the emr/ module scope, and terraform validate confirmed the validity of the EMR configuration. The terraform plan output demonstrates that the module correctly reads the existing foundation state - including the private subnet, VPC, and IAM instance profile - via Terraform data sources, and resolves the planned

output values for the S3 logs path (s3://25nplx-cisc886-bucket/emr-logs/) and S3 output path (s3://25nplx-cisc886-bucket/processed/). The plan indicates that four new resources are to be created, corresponding to the EMR cluster and its associated security groups.

```

}
+ name           = "25nplx-sg-emr-service"
+ name_prefix    = (known after apply)
+ owner_id       = (known after apply)
+ revoke_rules_on_delete = false
+ tags           = {
+   "Name"    = "25nplx-sg-emr-service"
+   "NetID"   = "25nplx"
+   "Project" = "cisc886"
+ }
+ tags_all       = {
+   "Name"    = "25nplx-sg-emr-service"
+   "NetID"   = "25nplx"
+   "Project" = "cisc886"
+ }
+ vpc_id         = "vpc-0962a0563e0d101bb"
}

Plan: 4 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ emr_cluster_id = (known after apply)
+ s3_logs_path   = "s3://25nplx-cisc886-bucket/emr-logs/"
+ s3_output_path = "s3://25nplx-cisc886-bucket/processed/"

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these
actions if you run "terraform apply" now.

```

Fig. (15): Terminal output of the sequential execution of terraform init, terraform validate, and terraform plan commands within the emr/ module directory. The output confirms provider initialisation, configuration validity, and that the planned changes affect only output variables (S3 log and output paths), with the EMR security groups and cluster being read from existing foundation state.

Execution of terraform apply -auto-approve within the EMR module triggered the sequential creation of the EMR master, core, and service security groups, followed by the provisioning of the Spark cluster resource. As illustrated in Fig. (16), Terraform reports continuous creation progress for aws_emr_cluster.spark_cluster at ten-second intervals, indicating that the cluster is undergoing bootstrapping and configuration. After approximately 3 minutes and 56 seconds, the apply operation completes successfully, reporting "1 added, 3 changed, 0 destroyed" and surfacing the cluster identifier j-TWNNM5XX1QN3 along with the configured S3 log and output paths. These outputs confirm that the EMR cluster has been successfully provisioned and is ready to accept the submitted PySpark preprocessing step.

```

PS D:\Queens_university\CISC_886_Cloud_Computing_M26\Project\files\Terraform_files\emr> terraform apply -auto-approve
data.aws_subnet.private: Reading...
data.aws_iam_instance_profile.emr_ec2_profile: Reading...
data.aws_iam_role.emr_service_role: Reading...
data.aws_vpc.main: Reading...
data.aws_iam_instance_profile.emr_ec2_profile: Read complete after 0s [id=A1PA2U3H6J3FVFHIQV0B5]
data.aws_iam_role.emr_service_role: Read complete after 0s [id=25nplx-emr-service-role]
data.aws_subnet.private: Read complete after 0s [id=subnet-099b009523fbee1b8]
data.aws_vpc.main: Read complete after 1s [id=vpc-0962a0563e0d101bb]
aws_security_group.emr_master: Refreshing state... [id=sg-0d2f4f6480e1e13cfb]
aws_security_group.emr_core: Refreshing state... [id=sg-00597b39b68e4541c]
aws_security_group.emr_service: Refreshing state... [id=sg-01d693eada54ca639]
aws_emr_cluster.spark_cluster: Refreshing state... [id=j-P1M92AX6283]

Note: Objects have changed outside of Terraform

Terraform detected the following changes made outside of Terraform since the last "terraform apply" which may
have affected this plan:

# aws_emr_cluster.spark_cluster has been deleted
- resource "aws_emr_cluster" "spark_cluster" {
  id           = "j-P1M92AX6283" -> null
  name         = "25nplx-emr"
  tags        = {
    "Name"    = "25nplx-emr"
    "NetID"   = "25nplx"
    "Project" = "cisc886"
  }
  # (20 unchanged attributes hidden)
  # (5 unchanged blocks hidden)
}

Unless you have made equivalent changes to your configuration, or ignored the relevant attributes using
ignore_changes, the following plan may include actions to undo or respond to these changes.

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
  create
  update in-place

Terraform will perform the following actions:

# aws_emr_cluster.spark_cluster will be created
+ resource "aws_emr_cluster" "spark_cluster" {
  applications = [
    "Hadoop",
    "Spark"
  ]

```

```

Changes to Outputs:
  - emr_cluster_id = "j-TWNNM5XX1QN3" -> (known after apply)
aws_security_group.emr_master: Modifications complete after 1s [id=sg-0d2f4f64801e13cfb]
aws_security_group.emr_service: Modifying... [id=sg-01dd963eda54ca639]
aws_security_group.emr_core: Modifying... [id=sg-00597b39b68e4541c]
aws_security_group.emr_service: Modifications complete after 1s [id=sg-01dd963eda54ca639]
aws_security_group.emr_core: Modifications complete after 1s [id=sg-00597b39b68e4541c]
aws_emr_cluster.spark_cluster: Creating...
aws_emr_cluster.spark_cluster: Still creating... [00m10s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [00m20s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [00m30s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [00m40s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [00m50s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [01m00s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [01m10s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [01m20s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [01m30s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [01m40s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [01m50s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [02m00s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [02m10s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [02m20s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [02m30s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [02m40s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [02m50s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [03m00s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [03m10s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [03m20s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [03m30s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [03m40s elapsed]
aws_emr_cluster.spark_cluster: Still creating... [03m50s elapsed]
aws_emr_cluster.spark_cluster: Creation complete after 3m56s [id=j-TWNNM5XX1QN3]

Apply complete! Resources: 1 added, 3 changed, 0 destroyed.

Outputs:
emr_cluster_id = "j-TWNNM5XX1QN3"
s3_logs_path = "s3://25nplx-cisc886-bucket/emr-logs/"
s3_output_path = "s3://25nplx-cisc886-bucket/processed/"

```

Fig. (16): Terminal output of terraform apply -auto-approve for the emr/ module, showing the creation of EMR security groups and the progressive provisioning of the Spark cluster (aws_emr_cluster.spark_cluster). The apply completes after approximately 3 minutes and 56 seconds, with the EMR cluster ID (j-TWNNM5XX1QN3) and S3 path outputs confirmed.

Following the successful completion of terraform apply, the Amazon EMR console was consulted to confirm the cluster's initial state. As shown in Fig. (17), the cluster 25nplx-emr is observed in the **Starting** state approximately 10 seconds after creation. The summary panel confirms the cluster's configuration: it runs on Amazon EMR version 6.15.0 with Hadoop 3.3.6 and Spark 3.4.1 as installed applications, comprises one primary node and two core nodes (m5.xlarge instances), and is configured with automatic termination upon completion of the last submitted step. Cluster logs are archived to the S3 path s3://25nplx-cisc886-bucket/emr-logs/, and termination protection is disabled, consistent with the ephemeral, cost-efficient cluster design adopted for this project.

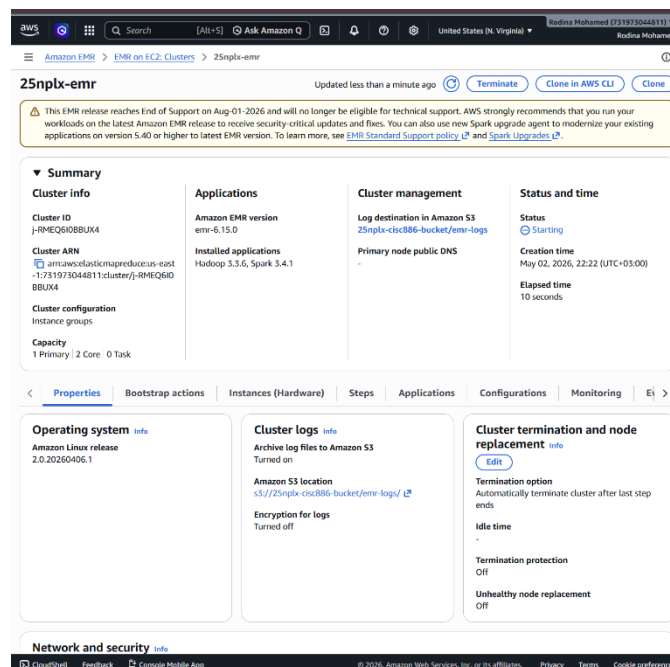


Fig. (17): Amazon EMR console view of the 25nplx-emr cluster in the Starting state, approximately 10 seconds after creation. The cluster summary displays the assigned cluster ID (j-RMEQ6I0BBLUX4), the Amazon EMR version (emr-6.15.0), installed applications (Hadoop 3.3.6, Spark 3.4.1), the cluster capacity (1 Primary, 2 Core, 0 Task nodes), and the configured termination policy (automatically terminate after last step ends).

Fig. (18) presents two sequential states of the 25nplx-emr cluster as observed through the Amazon EMR console. In the upper panel, the cluster is in the **Bootstrapping** state, indicating that the bootstrap.sh script is being executed across the cluster nodes to install required dependencies. In the lower panel, the cluster has transitioned to the **Running** state after approximately 4 minutes and 10 seconds, at which point the primary node's private DNS address (ip-10-0-2-158.ec2.internal) and SSH connectivity links become available. The Spark History Server and YARN Timeline Server persistent application UIs are also accessible in this state. The cluster's progression from Starting through Bootstrapping to Running confirms that the infrastructure configuration, network routing, and bootstrap actions were all executed successfully.

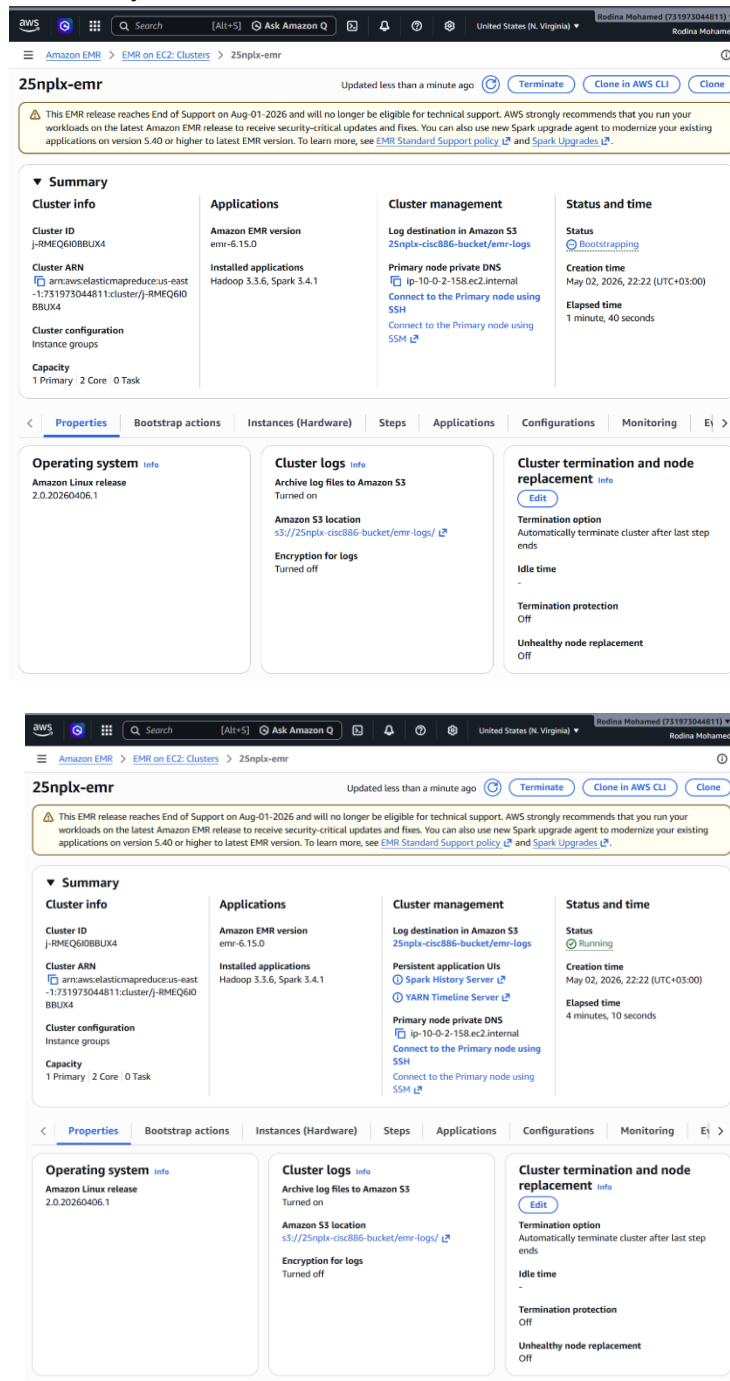


Fig. (18): Amazon EMR console view of the 25nplx-emr cluster transitioning through the Bootstrapping and Running states. The primary node's private DNS (ip-10-0-2-158.ec2.internal) and SSH connectivity links become available during the Running state, and the cluster elapsed time reaches approximately 4 minutes and 10 seconds.

Upon successful execution of the PySpark preprocessing step, the EMR cluster initiated automatic self-termination in accordance with the configured termination policy. As illustrated in Fig. (19), the Amazon EMR console captures the cluster 25nplx-emr sequentially transitioning through the **Terminating** state — with an elapsed time of 9 minutes and 58 seconds - before reaching the **Terminated** state at 22:33 (UTC+03:00) on 02 May 2026. The end-time timestamp and the reported elapsed duration confirm that the entire EMR job lifecycle, from cluster startup through bootstrapping, job execution, and self-termination, completed within approximately 10 minutes. This behaviour is consistent with the Terraform configuration, which set `keep_job_flow_alive_when_no_steps` to false, thereby ensuring no idle compute costs were incurred beyond the duration of the preprocessing job.

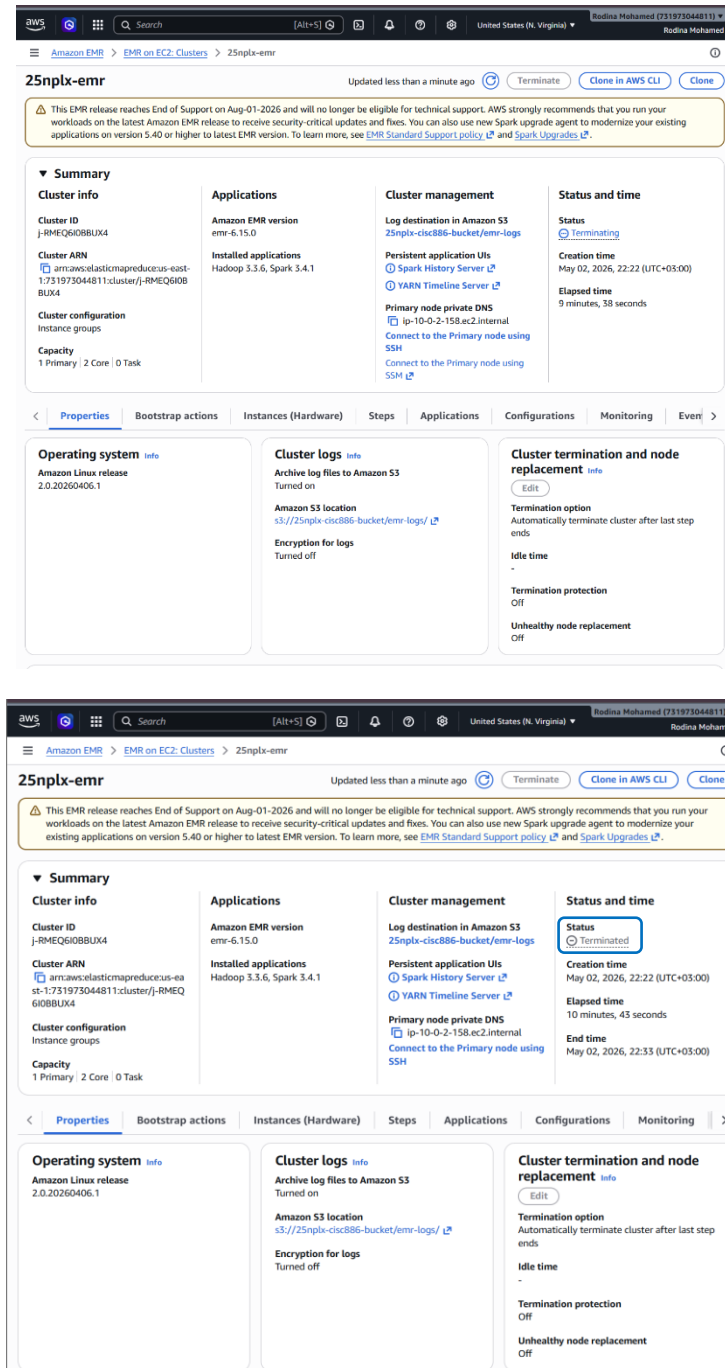


Fig. (19): Amazon EMR console view of the 25nplx-emr cluster transitioning through the Bootstrapping and Running states. The primary node's private DNS (ip-10-0-2-158.ec2.internal) and SSH connectivity links become available during the Running state, and the cluster elapsed time reaches approximately 4 minutes and 10 seconds.

Upon termination of the EMR cluster, the Amazon S3 console was examined to confirm that all expected output artefacts had been written successfully. As shown in Fig. (20), the 25nplx-cisc886-bucket contains four top-level entries: the emr-logs/ folder, which archives the cluster's application and step execution logs; the output/ folder, which contains all data and visualisation outputs generated by the PySpark preprocessing job; the scripts/ folder, housing the bootstrap and preprocessing scripts uploaded prior to cluster launch; and the source dataset file ubuntu_raw.parquet. The presence of all four entries confirms that the EMR job executed to completion and that its outputs - including the processed dataset partitions and generated figures - were correctly persisted to S3.

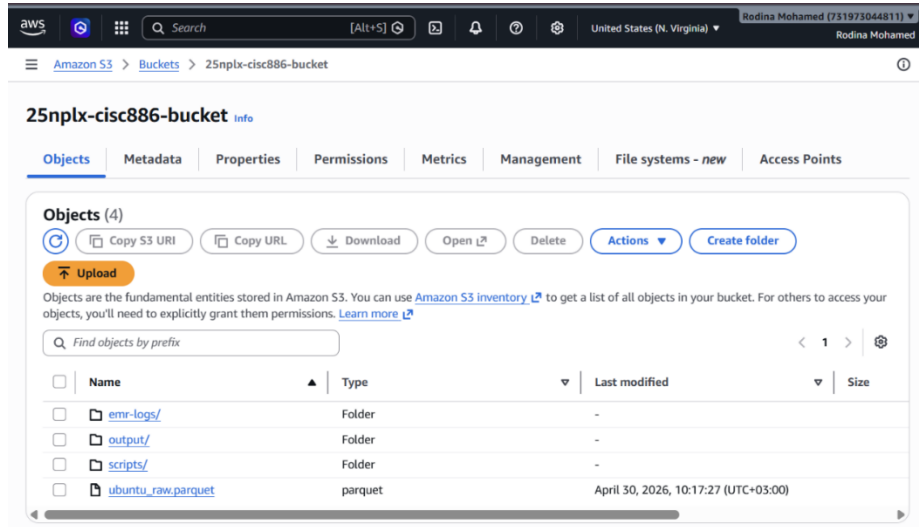


Fig. (20): Amazon S3 console view of the 25nplx-cisc886-bucket following the completion of the EMR preprocessing job, showing four top-level objects: the emr-logs/ folder (EMR execution logs), the output/ folder (preprocessed data and visualisation figures), the scripts/ folder (bootstrap and preprocessing scripts), and the ubuntu_raw.parquet source dataset file (735.5 MB)..

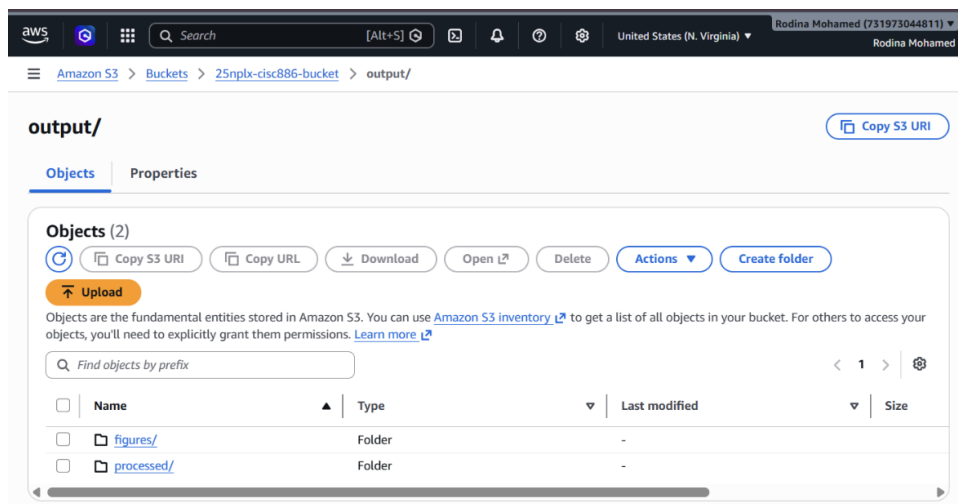


Fig. (21): Amazon S3 console view of the output/ prefix within 25nplx-cisc886-bucket, containing two sub-folders: figures/ (storing four exploratory data analysis PNG plots) and processed/ (storing the train, validation, and test dataset partitions in Parquet format).

The output/ prefix within the S3 bucket is structured into two sub-folders, as illustrated in Fig. 22. The figures/ directory contains four Portable Network Graphics (PNG) images generated by the PySpark preprocessing script as part of the exploratory data analysis: fig1_token_distribution.png, fig2_turns_per_dialogue.png, fig3_split_counts.png, and fig4_time_gap.png. These visualisations provide statistical characterisation of the processed dialogue corpus. The processed/ directory contains the output of the data partitioning step, which divided

the quality-filtered dataset into three stratified splits — train/, val/, and test/ — for use in the subsequent fine-tuning pipeline. This hierarchical storage organisation ensures a clear separation between analytical artefacts and model-ready training data.

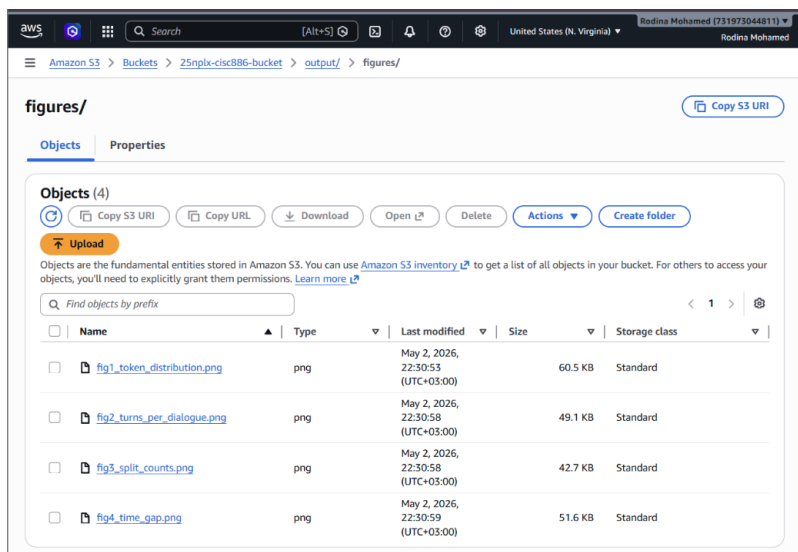


Fig. (23): Amazon S3 console view of the figures/ sub-folder within output/, listing four PNG visualisation files generated by the PySpark preprocessing job: fig1_token_distribution.png (60.5 KB), fig2_turns_per_dialogue.png (49.1 KB), fig3_split_counts.png (42.7 KB), and fig4_time_gap.png (51.6 KB), all stored in the S3 Standard storage class and generated on 02 May 2026.

The processed/ sub-folder, as depicted in Fig. (24), contains three directories corresponding to the train, validation, and test data splits produced by the PySpark preprocessing pipeline. Each directory contains nine Parquet part-files - reflecting the parallelism of the Spark write operation across the cluster's executor nodes - and a _SUCCESS zero-byte marker file, which Spark writes upon successful completion of a distributed write job. The train partition part-files are each approximately 334–335 MB in size, indicating a substantially larger training split consistent with standard machine learning practices. The validation and test partition files are each approximately 41–42 MB, reflecting their proportionally smaller allocations. The presence of _SUCCESS markers in all three directories confirms that the data partitioning step completed without error and that all split files are complete and uncorrupted.

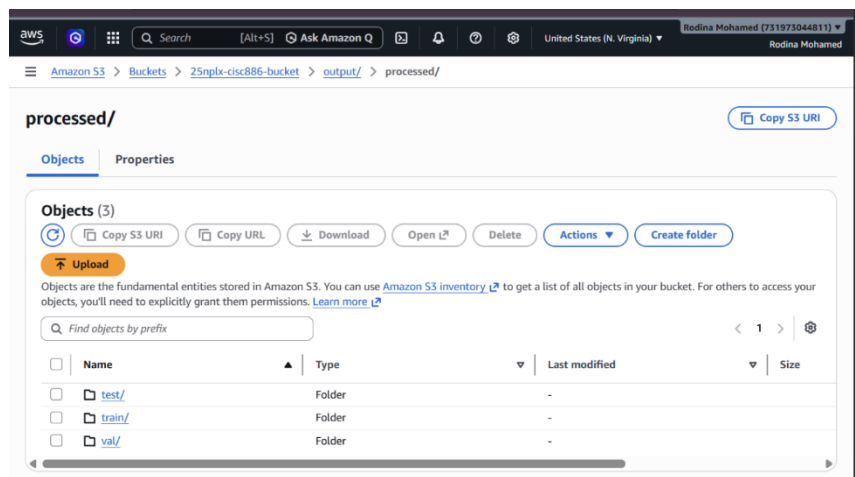


Fig. (24): Amazon S3 console view of the processed/ sub-folder within output/, showing three data partition directories: test/, train/, and val/, each containing nine Parquet part-files and a _SUCCESS marker confirming the successful completion of the Spark write operation.

Fig. (25) provides a detailed view of the train/ partition directory, listing all nine Parquet output files produced by the distributed Spark write operation. Each part-file — named according to the Spark convention part-NNNNN-<UUID>-c000.parquet — carries an approximate size of 334–335 MB, and all files share a consistent last-modified timestamp of 22:31 (UTC+03:00) on 02 May 2026, confirming that the write operation completed in a single, synchronised parallel step. The _SUCCESS marker file (0 bytes) is also present, serving as the canonical Spark indicator of a successfully completed distributed write. The training partition represents the largest data split and constitutes the primary input for the fine-tuning pipeline executed in Notebook 2.

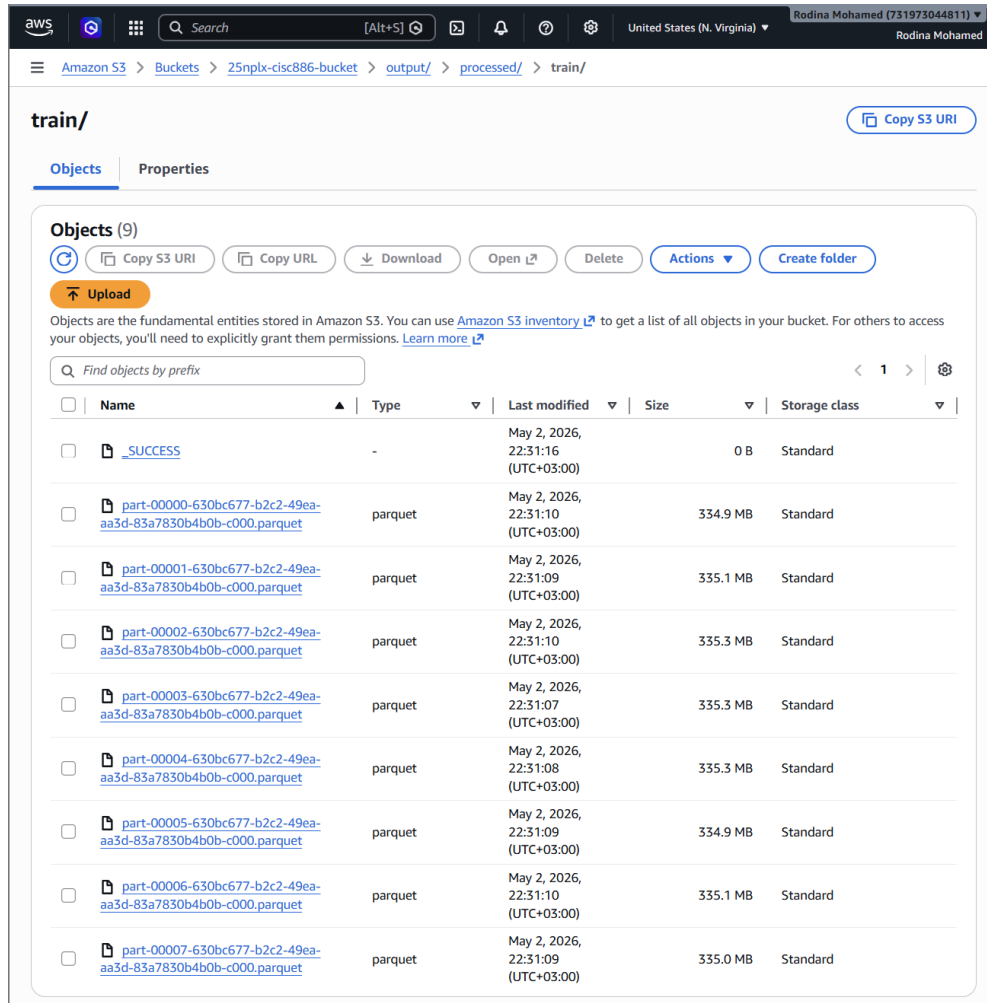


Fig. (25): Amazon S3 console view of the train/ sub-folder within processed/, listing nine Parquet part-files (each approximately 334–335 MB) and a _SUCCESS marker file, all generated on 02 May 2026 at 22:31 (UTC+03:00), confirming the successful distributed write of the training dataset partition.

Fig. (26) presents the directory listings for both the validation (val/) and test (test/) data partitions within the processed/ prefix, confirming the successful generation of both held-out evaluation splits. Each partition directory contains nine Parquet part-files with individual sizes in the range of 41–42 MB, and each is accompanied by a _SUCCESS marker file confirming write completion. The substantially smaller per-file sizes of these partitions relative to the training split reflect the proportional allocation of samples across the three sets, which is standard practice in supervised learning pipelines to reserve the majority of data for training while retaining sufficient samples for reliable evaluation. The validation partition is used for in-training performance monitoring, while the test partition serves as the final, unseen evaluation benchmark for comparing the baseline and fine-tuned model performance metrics.

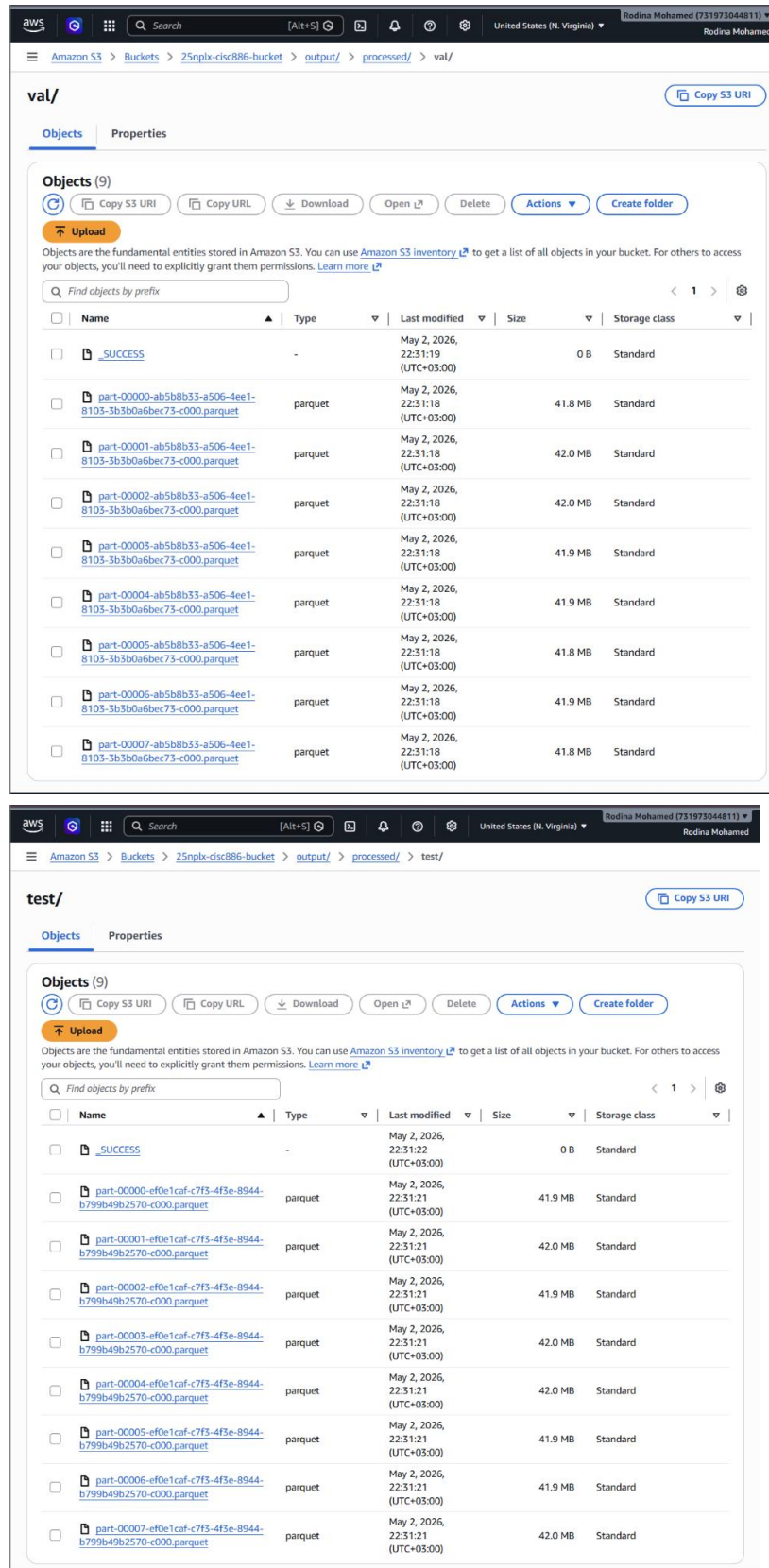
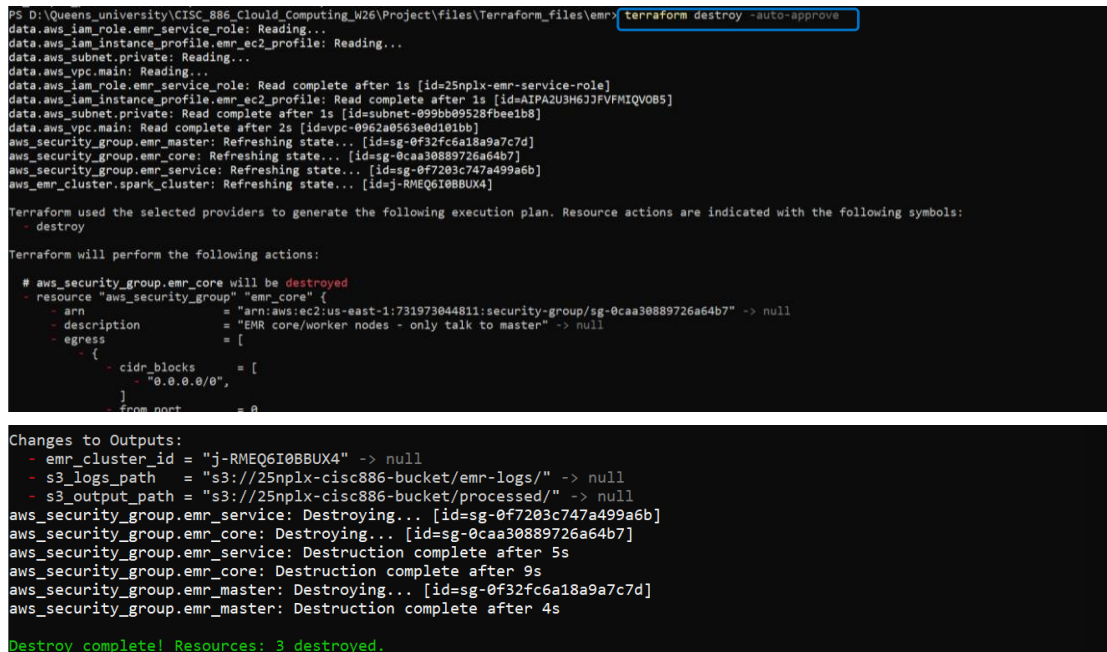


Fig. (26): Amazon S3 console views of the val/ and test/ sub-folders within processed/, each containing nine Parquet part-files (approximately 41–42 MB each) and a _SUCCESS marker, generated at 22:31 and 22:31–22:32 (UTC+03:00) respectively on 02 May 2026.

Following the completion of the EMR preprocessing job and confirmation of all expected S3 outputs, the EMR infrastructure was decommissioned by executing terraform destroy -auto-approve within the emr/ module directory. As illustrated in Fig. (27), Terraform read the current state of the three EMR security groups - emr_core, emr_master, and emr_service - and systematically destroyed each in dependency order: the emr_service security group was destroyed first (after 5 seconds), followed by emr_core (after 9 seconds) and emr_master (after 4 seconds). The final confirmation message, "Destroy complete! Resources: 3 destroyed," along with the nullification of all output variable values (cluster ID, S3 log path, and S3 output path), confirms that all EMR-specific cloud resources were cleanly removed and no residual charges would be incurred.



```
PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\emr> terraform destroy -auto-approve
data.aws_iam_role.emr_service_role: Reading...
data.aws_iam_instance_profile.emr_ec2_profile: Reading...
data.aws_subnet.private: Reading...
data.aws_vpc.main: Reading...
data.aws_iam_role.emr_service_role: Read complete after 1s [id=25nplx-emr-service-role]
data.aws_iam_instance_profile.emr_ec2_profile: Read complete after 1s [id=AIPA2U3H6JJFVFMQVOB5]
data.aws_subnet.private: Read complete after 1s [id=subnet-099bb09528fbee1b8]
data.aws_vpc.main: Read complete after 2s [id=vpc-0962a0563e0d101bb]
aws_security_group.emr_master: Refreshing state... [id=sg-0f32fc6a18a9a7c7d]
aws_security_group.emr_core: Refreshing state... [id=sg-0caa30889726a64b7]
aws_security_group.emr_service: Refreshing state... [id=sg-0f7203c747a499a6b]
aws_emr_cluster.spark_cluster: Refreshing state... [id=j-RMEQ6I0BBUX4]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
- destroy

Terraform will perform the following actions:

# aws_security_group.emr_core will be destroyed
resource "aws_security_group" "emr_core" {
  arn          = "arn:aws:ec2:us-east-1:731973044811:security-group/sg-0caa30889726a64b7" -> null
  description  = "EMR core/worker nodes - only talk to master" -> null
  egress       = [
    {
      cidr_blocks = [
        "0.0.0.0/0",
      ]
      from_port   = 0
    }
  ]
}

Changes to Outputs:
- emr_cluster_id = "j-RMEQ6I0BBUX4" -> null
- s3_logs_path   = "s3://25nplx-cisc886-bucket/emr-logs/" -> null
- s3_output_path = "s3://25nplx-cisc886-bucket/processed/" -> null
aws_security_group.emr_service: Destroying... [id=sg-0f7203c747a499a6b]
aws_security_group.emr_core: Destroying... [id=sg-0caa30889726a64b7]
aws_security_group.emr_service: Destruction complete after 5s
aws_security_group.emr_core: Destruction complete after 9s
aws_security_group.emr_master: Destroying... [id=sg-0f32fc6a18a9a7c7d]
aws_security_group.emr_master: Destruction complete after 4s

Destroy complete! Resources: 3 destroyed.
```

Fig. (27): Terminal output of terraform destroy -auto-approve for the emr/ module, confirming the sequential destruction of the three EMR security groups (emr_core, emr_master, and emr_service). The final output reports "Destroy complete! Resources: 3 destroyed," with all associated output

Notebook 1 – Data Preparation & Quality Scoring – Step Summary

Environment & Setup

- Install required packages: fastparquet, pandas, numpy, matplotlib.
- Import libraries and suppress warnings.

Load Raw Data

- Read three Parquet files (train.parquet, validation.parquet, test.parquet) containing raw dialogue turns.
- Columns: dialogue_id, context, response, time_gap_seconds.

Group Rows into Dialogues

- Group rows by dialogue_id and sort by time_gap_seconds.
- Keep only the last 5 turns of each dialogue (the resolution part).
- Format each dialogue into the Llama 3.2 Instruct chat template, including a system prompt.
- Filter dialogues to keep only those with 2-15 turns.

Scoring & Quality Filtering

- Score each dialogue on five dimensions:
 - Technical density (number of Linux commands, up to 40 points)
 - Resolution score (presence of success signals like “works”, “fixed”, up to 20 points)
 - Turn balance (3-8 turns is optimal, up to 15 points)
 - Response substance (average length of assistant replies, up to 15 points)

- Low-quality penalty (filler words like “lol”, “ok”, subtract up to 10 points)
- Composite score is the sum minus penalty, clamped to 0-100.
- Filter dialogues with composite score ≥ 40 (threshold adjustable).
- Save the filtered data (only the full_text column).

Visualisation & Diagnostics

- Plot histograms of composite scores, technical term hits, turns per dialogue, token lengths.
- Visualise filtered dialogue counts per split.
- Show correlation between technical hits and composite score.

Token Length Check

- Estimate token length (characters / 4) on a sample of 1000 training dialogues.
- Confirm that over 99% fit within the planned MAX_SEQ_LENGTH = 2048.

Sampling & Saving CSVs

- Randomly sample 30,000 dialogues for training, 3,000 for validation, 500 for testing.
- Save the sampled dialogues as train_ready.csv, valid_ready.csv, test_ready.csv.
- Each CSV contains only the full_text column (already formatted with Llama 3.2 chat template).

Verification

- Load each CSV to confirm no null values and correct format.

Key Parameters (as used)

- QUALITY_THRESHOLD = 40 – minimum composite score to keep a dialogue.
- MAX_TURNS = 5 – keep only the last 5 turns per dialogue.
- MAX_TRAIN = 30000, MAX_VALID = 3000, MAX_TEST = 500 – sample sizes.
- Scoring weights: technical density (40), resolution (20), turn balance (15), substance (15), penalty (-10).

Expected Outputs

- train_ready.csv (~30,000 dialogues, ~35 MB)
- valid_ready.csv (~3,000 dialogues, ~2.8 MB)
- test_ready.csv (~500 dialogues, ~0.5 MB)
- dialogue_quality_scores.png – diagnostic plots.

Important Notes

- The raw Parquet files may be split into parts; they must be concatenated into single train.parquet, validation.parquet, test.parquet before running.
- The scoring function is customisable (Linux commands list, weights, thresholds).
- Only the full_text column is saved to keep CSV sizes small and simplify training in Notebook 2.

Notebook 2 – Fine-Tuning + HuggingFace Push – Step Summary:

Environment & Setup

- Create a Python virtual environment (recommended).
- Install required packages: Unsloth, PyTorch, Transformers, Datasets, TRL, ROUGE-score, scikit-learn, pandas, matplotlib.
- Verify GPU availability and CUDA.

Data Preparation

- Ensure CSV files from Notebook 1 (train_ready.csv, valid_ready.csv, test_ready.csv) are present.
- Load CSVs into Hugging Face Dataset objects (the full_text column contains formatted Llama-3.2 chat templates).

Model Loading

- Load Llama 3.2 3B Instruct with 4-bit QLoRA using Unsloth.
- Attach LoRA adapters (rank = 16, alpha = 32, target modules = attention and MLP layers).
- Enable gradient checkpointing and rank-stabilised LoRA.

Baseline Evaluation

- Run inference on 100 test samples using the base model (before fine-tuning).
- Compute ROUGE-L F1 score, relevance rate (keyword-based), and classification metrics (relevance detection).
- Record evaluation time.

Fine-Tuning

- Train the model for 1000 steps with a cosine learning rate schedule.
- Training batch size: 2, gradient accumulation steps: 4, learning rate: 2e-4.
- Evaluate every 100 steps on validation set; save checkpoints.
- Load the best checkpoint (step 800, selected by lowest validation loss).

Fine-Tuned Evaluation

- Evaluate the fine-tuned model on the same 100 test samples.
- Compute ROUGE-L F1, relevance rate, accuracy, precision, recall, F1 score.
- Compare results with the base model.

Outputs & Saving

- Save LoRA adapter locally (llama3b_ubuntu_lora/, ~115 MB).
- (Optional) Export to GGUF format (4-bit quantized, ~2 GB) for use with Ollama.
- (Optional) Push LoRA adapter and GGUF model to HuggingFace Hub.

Usage of Trained Model

- Generate responses inside the notebook using a helper function.
- Load the saved LoRA adapter on top of the base Llama 3.2 model.
- (Optional) Run the GGUF model with Ollama on EC2 or locally.

Key Configuration Parameters (as used)

- Maximum sequence length: 2048 tokens.
- 4-bit loading enabled.
- LoRA rank (r) = 16, LoRA alpha = 32.
- Training steps: 1000.
- Learning rate: 2e-4.
- Batch size: 2 (with gradient accumulation steps = 4 $\rightarrow$ effective batch size = 8).

Evaluation Metrics (Reference)

- Base model: ROUGE-L F1 = 0.061, relevance rate = 99.0%, accuracy = 54.0%.
- Fine-tuned model (step 800): ROUGE-L F1 = 0.197, relevance rate = 51.0%, accuracy = 64.0%.

Expected Output Files

- checkpoints/ – training checkpoints.
- llama3b_ubuntu_lora/ – LoRA adapter files.
- training_curve.png – loss plot.
- llama3b_ubuntu_gguf_gguf/ – GGUF quantized model (optional).

Important Notes

- Start with a fresh kernel to free RAM.
- The notebook uses subsampled data (10k train, 2k validation) for speed; increase for production.
- LoRA adapter is small (~115 MB), easy to share and deploy.
- GGUF export is optional but recommended for Ollama deployment on EC2.

The deployment infrastructure, comprising the Bastion host and the LLM inference EC2 instance, was provisioned by executing `terraform apply -auto-approve` within the deployment/ module directory. As shown in Fig. (28), Terraform created four resources: the bastion security group (completed after 4 seconds), the EC2 deployment security group (completed after 4 seconds), the bastion EC2 instance (completed after 15 seconds), and the LLM EC2 deployment instance (completed after 14 seconds). The outputs section exposes the key connectivity information: the

bastion host's public IP address (34.205.87.101), the LLM EC2 instance's private IP address (10.0.2.163), and the pre-formatted SSH commands for establishing Ollama API tunnels and Open WebUI port-forwarding tunnels. These outputs allow direct copy-paste execution of the required SSH commands without manual IP substitution.

```
PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\deployment> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\deployment> terraform validate
Success! The configuration is valid.

PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\deployment> terraform plan
data.aws_subnet.public: Reading...
data.aws_vpc.main: Reading...
data.aws_subnet.private: Reading...
data.aws_iam_instance_profile.ec2_deployment_profile: Reading...
data.aws_iam_instance_profile.ec2_deployment_profile: Read complete after 1s [id=AIPA2U3H6JJFUKMXXTLFH]
data.aws_subnet.private: Read complete after 1s [id=subnet-099bb09528fbae1b8]
data.aws_subnet.public: Read complete after 1s [id=subnet-06ab8b608fd0becd0]
data.aws_vpc.main: Read complete after 2s [id=vpc-0962a0563e0d101bb]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.bastion will be created
+ resource "aws_instance" "bastion" {
  + ami                    = "ami-0e86e20dae9224db8"
  + arn                   = (known after apply)
  + associate_public_ip_address = true
  + availability_zone      = (known after apply)
  + cpu_core_count         = (known after apply)
  + cpu_threads_per_core   = (known after apply)
  + disable_api_stop       = (known after apply)
  + disable_api_termination = (known after apply)
}
```

```
PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\deployment> terraform apply -auto-approve
data.aws_subnet.public: Reading...
data.aws_vpc.main: Reading...
data.aws_iam_instance_profile.ec2_deployment_profile: Reading...
data.aws_subnet.private: Reading...
data.aws_iam_instance_profile.ec2_deployment_profile: Read complete after 1s [id=AIPA2U3H6JJFUKMXXTLFH]
data.aws_subnet.private: Read complete after 1s [id=subnet-099bb09528fbae1b8]
data.aws_subnet.public: Read complete after 1s [id=subnet-06ab8b608fd0becd0]
data.aws_vpc.main: Read complete after 1s [id=vpc-0962a0563e0d101bb]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_instance.bastion will be created
+ resource "aws_instance" "bastion" {
  + ami                    = "ami-0e86e20dae9224db8"
  + arn                   = (known after apply)
  + associate_public_ip_address = true
  + availability_zone      = (known after apply)
  + cpu_core_count         = (known after apply)
  + cpu_threads_per_core   = (known after apply)
}
```

```
aws_security_group.bastion: Creating...
aws_security_group.bastion: Creation complete after 4s [id=sg-086e825c9368a260b]
aws_security_group.ec2_deployment: Creating...
aws_instance.bastion: Creating...
aws_security_group.ec2_deployment: Creation complete after 4s [id=sg-0025757dcd3118954]
aws_instance.deployment: Creating...
aws_instance.bastion: Still creating... [00m10s elapsed]
aws_instance.deployment: Still creating... [00m10s elapsed]
aws_instance.bastion: Creation complete after 15s [id=i-0d3f7c035dde7362f]
aws_instance.deployment: Creation complete after 14s [id=i-0d9d96de498ce5e2b]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:
bastion_public_ip = "34.205.87.101"
curl_test_command = "curl http://localhost:11434/api/generate -d '{\"model\": \"25nplx-1lama3b\", \"prompt\": \"How do I check disk usage in Ubuntu?\", \"stream\": false}'"
ec2_private_ip = "10.0.2.163"
ollama_tunnel_command = "ssh -i 25nplx-keypair.pem -L 11434:10.0.2.163:11434 ubuntu@34.205.87.101"
openwebui_tunnel_command = "ssh -i 25nplx-keypair.pem -L 8080:10.0.2.163:8080 ubuntu@34.205.87.101"
ssh_to_bastion = "ssh -i 25nplx-keypair.pem ubuntu@34.205.87.101"
ssh_to_llm_via_bastion = "ssh -i 25nplx-keypair.pem -l ubuntu@34.205.87.101 ubuntu@10.0.2.163"
```

Fig. (28): Terminal output of terraform apply -auto-approve for the deployment/ module, confirming the creation of four resources: the bastion security group, the EC2 deployment security group, the bastion EC2 instance, and the LLM EC2 deployment instance. The outputs section provides the bastion public IP (34.205.87.101), EC2 private IP (10.0.2.163), and the Ollama tunnel and OpenWebUI tunnel SSH commands.

SSH clients on Windows enforce strict requirements on the permissions of private key files, rejecting keys that are accessible to accounts other than the connecting user. To satisfy this requirement, the private key file was renamed with the .pem extension, and its access control list was modified using the `icacls` utility. As illustrated in Fig. (29), the four-step permission configuration - removing inherited permissions (`/inheritance:r`), granting read-only access exclusively to the current user (`/grant:r "$($env:USERNAME):(R)"`), and explicitly removing both `NT AUTHORITY\Authenticated Users` and `Everyone` — was applied successfully to the key file, with each command reporting "Successfully processed 1 files; Failed processing 0 files." This permission configuration ensures that the SSH client accepts the private key file and permits authenticated access to the bastion host.

- # Store the file path
- \$keyPath = "D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities\25nplx-keypair.pem"
- # Remove all existing permissions
`icacls $keyPath /inheritance:r`
- # Grant only your user read access
`icacls $keyPath /grant:r "$($env:USERNAME):(R)"`
- # Remove Authenticated Users and Everyone
`icacls $keyPath /remove "NT AUTHORITY\Authenticated Users"`
`icacls $keyPath /remove "Everyone"`

```
PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\deployment> # Store the file path
>> $keyPath = "D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities\25nplx-keypair.pem"
>>
>> # Remove all existing permissions
>> icacls $keyPath /inheritance:r
>>
>> # Grant only your user read access
>> icacls $keyPath /grant:r "$($env:USERNAME):(R)"
>>
>> # Remove Authenticated Users and Everyone
>> icacls $keyPath /remove "NT AUTHORITY\Authenticated Users"
>> icacls $keyPath /remove "Everyone"
processed file: D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities\25nplx-keypair.pem
Successfully processed 1 files; Failed processing 0 files
processed file: D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities\25nplx-keypair.pem
Successfully processed 1 files; Failed processing 0 files
processed file: D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities\25nplx-keypair.pem
Successfully processed 1 files; Failed processing 0 files
processed file: D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities\25nplx-keypair.pem
Successfully processed 1 files; Failed processing 0 files
```

Fig. (29): PowerShell terminal output confirming the successful application of restrictive file permissions to the SSH private key file `25nplx-keypair.pem` using `icacls`. Each of the four permission modification commands — removing inheritance, granting read-only access to the current user, and removing `Authenticated Users` and `Everyone` - is confirmed as having processed one file successfully.

Successful SSH connectivity to the bastion host was established by executing the SSH command (`ssh -i "D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities\25nplx-keypair.pem" ubuntu@bastion_public_ip`) with the configured .pem private key, targeting the bastion's public IP address (34.205.87.101). As shown in Fig. (30), the connection is accepted and the Ubuntu 24.04 LTS welcome banner is displayed, confirming that the bastion EC2 instance is operational and reachable from the local development machine. The system information panel reports a low system load (0.0), 23.1% disk utilisation of a 6.71 GB volume, 24% memory usage, and one logged-in user, consistent with a freshly provisioned instance. The successful connection validates the correct configuration of the bastion security group rules, the key pair association, and the public subnet routing through the Internet Gateway.

```

PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\deployment> ssh -i "D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities\25nplx-keypair.pem" ubuntu@34.205.87.101
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.0-1012-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon May  4 08:56:30 UTC 2026

System load:  0.0           Temperature:   -273.1 C
Usage of /:   23.1% of 6.71GB Processes:      110
Memory usage: 24%          Users logged in: 1
Swap usage:   0%           IPv4 address for ens5: 10.0.1.32

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Mon May  4 07:33:33 2026 from 41.218.155.81
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-10-0-1-32:~$

```

Fig. (30): Terminal output confirming SSH connectivity to the Bastion host (ubuntu@34.205.87.101) using the configured private key. The Ubuntu 24.04 LTS system information banner is displayed, reporting system load, memory usage, disk usage, and the number of logged-in users at the time of connection (Monday, 04 May 2026, 08:56 UTC).

To enable SSH access from the bastion host to the private LLM EC2 instance - which resides in the private subnet and is not directly reachable from the public internet - the SSH private key file was transferred to the bastion host using the secure copy (scp) command (scp -i "D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities\25nplx-keypair.pem" "D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities\25nplx-keypair.pem" ubuntu@ bastion_public_ip:~/.ssh/25nplx-keypair.pem). As illustrated in Fig. (31), the 3,389-byte 25nplx-keypair.pem file was transferred to the path ~/.ssh/25nplx-keypair.pem on the bastion host at a rate of 24.9 KB/s, completing in under one second at 100% transfer completion. Following the transfer, the key's permissions on the bastion host were set to owner-read-only (chmod 400 ~/.ssh/25nplx-keypair.pem) before executing the subsequent SSH command to connect to the private EC2 instance at 10.0.2.163.

```

PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\deployment> scp -i "D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities\25nplx-keypair.pem" "D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities\25nplx-keypair.pem" ubuntu@34.205.87.101:~/.ssh/25nplx-keypair.pem
25nplx-keypair.pem                                100% 3389    24.9KB/s   00:00

```

Fig. (31): Terminal output confirming the secure copy (scp) transfer of the SSH private key file (25nplx-keypair.pem, 3,389 bytes) from the local machine to the bastion host's ~/.ssh/ directory at a transfer rate of 24.9 KB/s, enabling subsequent SSH jump-host connectivity from the bastion to the private EC2 instance.

From within the bastion host session, SSH connectivity to the private LLM EC2 instance was established using the transferred private key, targeting the private IP address 10.0.2.163. As shown in Fig. (32), the connection is accepted and the Ubuntu 24.04 LTS system banner is displayed. The reported system metrics indicate a substantially larger disk volume (47.39 GB) compared to the bastion host, reflecting the larger instance type (t3.2xlarge) required to host the quantised LLM and Docker-based Open WebUI container. The 35% disk utilisation and 164 active processes suggest that the user-data bootstrap script - which installs Ollama, pulls the fine-tuned GGUF model from HuggingFace, and starts the Open WebUI Docker container - had already completed prior to the SSH connection. A system restart notification is also displayed, which is a standard artefact of automated package updates applied by the bootstrap script.

```

ubuntu@ip-10-0-1-32:~$ chmod 400 ~/.ssh/25nplx-keypair.pem
ubuntu@ip-10-0-1-32:~$ ssh -i ~/.ssh/25nplx-keypair.pem ubuntu@10.0.2.163
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-1012-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon May  4 08:59:47 UTC 2026

System load:  0.0           Temperature:   -273.1 C
Usage of /:   35.0% of 47.39GB Processes:      164
Memory usage: 6%           Users logged in: 1
Swap usage:   0%           IPv4 address for ens5: 10.0.2.163

Expanded Security Maintenance for Applications is not enabled.

16 updates can be applied immediately.
3 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

1 additional security update can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

*** System restart required ***
Last login: Mon May  4 08:59:47 2026 from 10.0.1.32
ubuntu@ip-10-0-2-163:~$

```

Fig. (32): Terminal output confirming successful SSH connectivity from the Bastion host to the private LLM EC2 instance (ubuntu@10.0.2.163). The Ubuntu 24.04 LTS system information banner reports 35% disk utilisation of a 47.39 GB volume, 6% memory usage, and 164 active processes, indicating that the instance's bootstrap provisioning had completed prior to connection.

The bootstrap script's execution status was verified by inspecting the user-data log file using `sudo tail -f /var/log/user-data.log`. As illustrated in Fig. 33, the log records the progressive download and extraction of the Open WebUI Docker image layers - identified by their SHA256 digest hashes - culminating in the "Setup complete!" message that marks the successful conclusion of all bootstrap actions. This confirmation indicates that Ollama has been installed and started as a systemd service, the fine-tuned GGUF model has been pulled from HuggingFace Hub and registered with Ollama, and the Open WebUI Docker container has been successfully launched and is bound to port 8080 on the private EC2 instance. The "Setup complete!" message serves as the definitive indicator that the deployment is ready for service verification. (hint: click on ctrl+C to get out)

```

ubuntu@ip-10-0-2-163:~$ sudo tail -f /var/log/user-data.log
0567bba6946b: Download complete
0567bba6946b: Pull complete
ade6fdd14ee3: Pull complete
4524391b1074: Pull complete
801abe499289: Pull complete
e221127a7c39: Pull complete
Digest: sha256:c2e4723fdbca5de8f9f0529e22b78acf5bc312a88da730bed88860063d028fe8
Status: Downloaded newer image for ghcr.io/open-webui/open-webui:main
2f03e438a17f0bfd35c75ef4164cbb0f6804f7f402f2e3d967bb9359b1881b9f
Setup complete!

```

Fig. (33): Terminal output of `sudo tail -f /var/log/user-data.log` on the private EC2 instance, confirming the successful completion of the bootstrap provisioning script. The log shows the sequential Docker image layer pulls for the Open WebUI container, culminating in the "Setup complete!" confirmation message.

The operational status of the Ollama inference server was verified using two commands. First, `systemctl status ollama` confirmed that the Ollama service is in the **active (running)** state, having been started at 07:35:38 UTC on 04 May 2026, with a main process ID of 22054 and a resident memory footprint of 2.1 GB (peak 4.3 GB). Second, `curl http://localhost:11434/api/tags` queried the Ollama REST API to enumerate the registered model variants. As shown in Fig. (34), the API response lists two available GGUF models: the base `hf.co/Rodina222/ubuntu-dialogue-llama3b-gguf:latest` and the fine-tuned `chatbot 25nplx-ubuntu-assistant-llama3b-chatbot:latest`, both in the `q4_k_m` quantisation format and of approximately the same parameter size (3.21B). The presence of both models confirms successful model registration with Ollama. (hint: click on ctrl+C to exit)

```

ubuntu@ip-10-0-2-163:~$ systemctl status ollama
curl http://localhost:11434/api/tags
* ollama.service - Ollama Service
   Loaded: loaded (/etc/systemd/system/ollama.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-05-04 07:35:38 UTC; 1h 39min ago
     Main PID: 22054 (ollama)
        Tasks: 23 (limit: 38049)
       Memory: 2.1G (peak: 4.3G)
          CPU: 29min 38.871s
      CGroup: /system.slice/ollama.service
              └─22054 /usr/local/bin/ollama serve

May 04 09:02:58 ip-10-0-2-163 ollama[22054]: llama_context: graph nodes = 875
May 04 09:02:58 ip-10-0-2-163 ollama[22054]: llama_context: graph splits = 1
May 04 09:02:58 ip-10-0-2-163 ollama[22054]: time=2026-05-04T09:02:58.890Z level=INFO source=server.go:1432 msg="llama runner s
May 04 09:02:58 ip-10-0-2-163 ollama[22054]: time=2026-05-04T09:02:58.890Z level=INFO source=sched.go:561 msg="loaded runners" s
May 04 09:02:58 ip-10-0-2-163 ollama[22054]: time=2026-05-04T09:02:58.890Z level=INFO source=server.go:1385 msg="waiting for llb
May 04 09:02:58 ip-10-0-2-163 ollama[22054]: time=2026-05-04T09:02:58.891Z level=INFO source=server.go:1432 msg="llama runner s
May 04 09:03:00 ip-10-0-2-163 ollama[22054]: [GIN] 2026/05/04 - 09:03:00 | 200 | 4.450149311s | 127.0.0.1 | POST | "/"
May 04 09:11:29 ip-10-0-2-163 ollama[22054]: [GIN] 2026/05/04 - 09:11:29 | 200 | 521.65µs | 127.0.0.1 | GET | "/"
May 04 09:12:57 ip-10-0-2-163 ollama[22054]: [GIN] 2026/05/04 - 09:12:57 | 200 | 25.086µs | 127.0.0.1 | HEAD | "/"
May 04 09:12:57 ip-10-0-2-163 ollama[22054]: [GIN] 2026/05/04 - 09:12:57 | 200 | 415.081µs | 127.0.0.1 | GET | "/"

{"models":[{"name":"hf.co/Rodina222/ubuntu-dialogue-llama3b-gguf:latest","model":"hf.co/Rodina222/ubuntu-dialogue-llama3b-gguf:late
st","modified_at":"2026-05-04T07:35:56.093985238Z","size":2019379442,"digest":"e19d24687884b8ad74e38a903ca0ef99862dbbe35462f1
8f6b94ea05794b5ea9","details":{"parent_model":"","format":"gguf","family":"llama","families":["llama"],"parameter_size":"3.21B",
"quantization_level":"unknown"}},{name:"25nplx-ubuntu-assistant-llama3b-chatbot:latest","model":"25nplx-ubuntu-assistant-llama
3b-chatbot:latest","modified_at":"2026-05-04T07:35:56.15498564Z","size":2019379501,"digest":"d181f7c4c1d45a738b4c7e1361e2bfc127c
262d7ba5787f6baa17150d1c05fd3","details":{"parent_model":"","format":"gguf","family":"llama","families":["llama"],"parameter_siz
e":"3.2B","quantization_level":"Q4_K_M"}}]}
ubuntu@ip-10-0-2-163:~$

```

Fig. (34): Terminal output confirming that the Ollama service is active and running on the private EC2 instance, as reported by `systemctl status ollama`, and that two GGUF model variants are registered and available, as returned by `curl http://localhost:11434/api/tags`. The service has been active for 1 hour and 39 minutes, with a memory footprint of 2.1 GB (peak 4.3 GB).

A functional test of the deployed fine-tuned model was performed by submitting a POST request to the Ollama inference API endpoint using `curl`, with the model identifier set to `25nplx-ubuntu-assistant-llama3b-chatbot` and the prompt "How do I check disk usage in Ubuntu?" As illustrated in Fig. (35), the model generates the response "sudo df -h", which is a concise and technically correct answer to the posed Ubuntu technical query. The response metadata reports "done_reason": "stop", indicating that the model produced a complete response. The total_duration field records approximately 42.8 seconds for the first cold-start inference, which includes model loading time and is expected to decrease substantially for subsequent warm queries. This test confirms that the fine-tuned model is correctly loaded, functional, and producing domain-appropriate technical responses via the Ollama API. Using command:

```

curl http://localhost:11434/api/generate -d '{
  "model": "25nplx-ubuntu-assistant-llama3b-chatbot",
  "prompt": "How do I check disk usage in Ubuntu?",
  "stream": false}'

```

```

ubuntu@ip-10-0-2-163:~$ curl http://localhost:11434/api/generate -d '{
  "model": "25nplx-ubuntu-assistant-llama3b-chatbot",
  "prompt": "How do I check disk usage in Ubuntu?",
  "stream": false}'
{"model":" 25nplx-ubuntu-assistant-llama3b-chatbot","created_at":"2026-05-04T09:17:05.416920406Z","response":"sudo df -h","done
":true,"done_reason":"stop","context":[128006,9125,128007,271,2675,527,264,11190,36060,13312,1862,18328,13,128009,128006,882,12
8007,271,4438,656,358,1817,13668,10648,304,36060,30,128009,128006,78191,128007,271,19413,6907,482,71],"total_duration":42837896
22,"load_duration":2492319386,"prompt_eval count":33,"prompt_eval duration":1426547951,"eval count":5,"eval duration":349985552

```

Fig. (35): Terminal output of a `curl` request to the Ollama `/api/generate` endpoint, querying the fine-tuned model `25nplx-ubuntu-assistant-llama3b-chatbot` with the prompt "How do I check disk usage in Ubuntu?". The model responds with "sudo df -h" and the response metadata confirms a total inference duration of approximately 42.8 seconds on the first cold-start invocation.

Docker permissions were enabled on the private EC2 instance by adding the `ubuntu` user to the Docker group, and successful configuration was confirmed using `sudo usermod -aG docker ubuntu`, `newgrp docker`, and `docker ps` command Fig. (36).

```

ubuntu@ip-10-0-2-76:~$ sudo usermod -aG docker ubuntu
newgrp docker
docker ps

```

Fig. (36): Verification of Docker installation and active containers on the private EC2 instance after granting the `ubuntu` user permission to execute Docker commands without elevated privileges.

The Open WebUI graphical chat interface was accessed by establishing an SSH port-forwarding tunnel from the local machine to the private EC2 instance through the bastion host, mapping local port 8080 to port 8080 on the private instance. Upon navigating to <http://localhost:8080/> in a local browser, the Open WebUI interface was rendered successfully, as shown in Fig. (38). The model selector displays `hf.co/Rodina222/ubuntu-dialogue-llama3b-gguf:latest` as the currently selected model, confirming that the HuggingFace-hosted GGUF variant of the fine-tuned model is registered and accessible within the interface. The welcome prompt panel is visible, presenting suggested conversation starters. This confirms the end-to-end accessibility of the deployed model through a user-friendly web interface, accessible from a standard browser via the SSH tunnel without exposing any ports directly to the public internet

SSH Tunnel Command in new terminal:

```
ssh -i "D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities\25nplx-keypair.pem" -L 8080:ec2_private_ip:8080 ubuntu@bastion_public_ip -N
```

```
PS C:\Users\aeasa> ssh -i "D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\utilities\25nplx-keypair.pem" -L 8080:10.0.2.76:8080 ubuntu@54.160.27.80 -N
```

Fig. (37): SSH tunneling command executed on the local machine to securely forward traffic from port 8080 to the private EC2 instance via the bastion host, enabling browser-based access to Open WebUI.

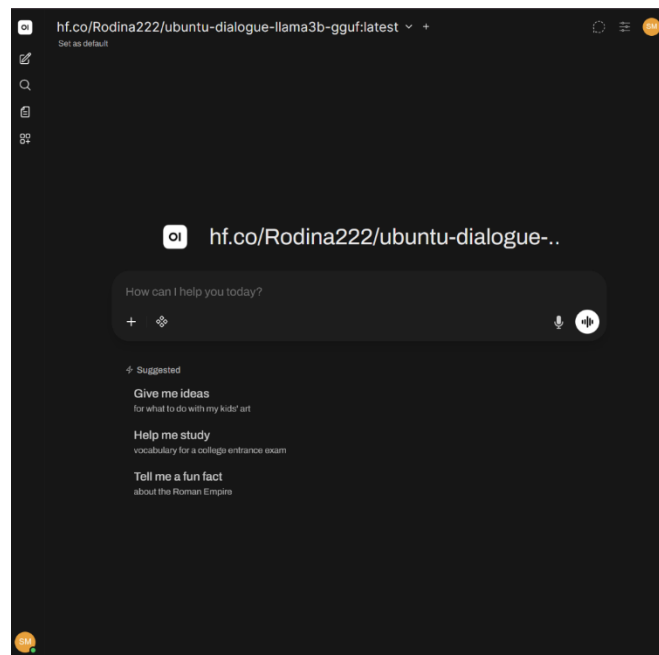


Fig. (38): Open WebUI interface accessible at <http://localhost:8080/> via SSH port forwarding, displaying the model selector with `hf.co/Rodina222/ubuntu-dialogue-llama3b-gguf:latest` selected as the active model. The chat interface shows the standard Open WebUI welcome prompt, confirming successful deployment and browser-based accessibility of the fine-tuned LLM.

Fig. (39) presents a live interaction session with the deployed fine-tuned model through the Open WebUI interface, demonstrating the model's domain-adapted response behaviour across three distinct Ubuntu technical queries. In response to the query "How do I check disk usage in Ubuntu?", the model responds "Use the 'du' command" - a correct and domain-appropriate answer. For the query "How do I list all running processes in Ubuntu?", the model responds "use the 'ps' command, or try top", accurately referencing two standard Unix process inspection utilities. For the query "How do I give a user sudo privileges?", the model responds "You need to create an entry for that user in /etc/sudoers. There's howto guides on the web site." - a contextually appropriate answer that reflects the informal conversational style characteristic of the Ubuntu Dialogue Corpus on which the model was fine-tuned. Collectively, these interactions demonstrate that the fine-tuning procedure has successfully adapted the base Llama 3.2 3B Instruct model to generate relevant, concise, and technically grounded responses to Ubuntu system administration queries.

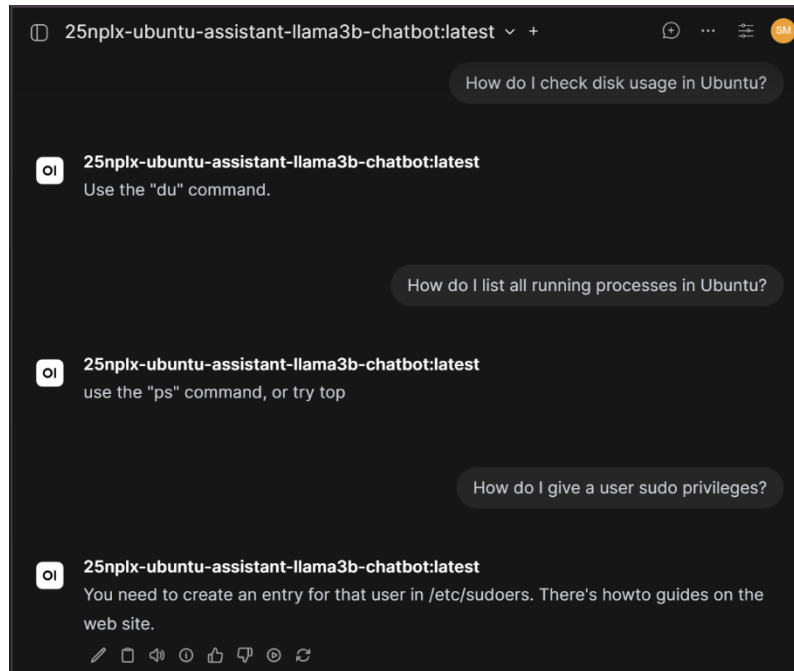


Fig. (39): Open WebUI chat session with the deployed 25nplx-ubuntu-assistant-llama3b-chatbot:latest model, demonstrating responses to three Ubuntu-specific technical queries: disk usage checking ("Use the 'du' command"), listing running processes ("use the 'ps' command, or try top"), and granting sudo privileges ("You need to create an entry for that user in /etc/sudoers"). The responses confirm that the fine-tuned model produces concise and contextually appropriate technical answers.

After completing the demonstration, all active sessions were terminated by first closing the SSH tunnel using Ctrl + C, followed by exiting the Ubuntu bastion and private EC2 terminals using the exit command until all remote sessions were fully closed. Following the conclusion of the deployment demonstration, the deployment infrastructure was decommissioned by executing terraform destroy -auto-approve within the deployment/ module directory. As illustrated in Fig. (40), Terraform resolved the current resource states — including both EC2 instances and their associated security groups — before initiating sequential destruction. The bastion EC2 instance was destroyed first, followed by the LLM deployment instance, which required approximately 41 seconds to terminate. The EC2 deployment and bastion security groups were then destroyed within 1 second each. The outputs panel confirms the nullification of all output variables (bastion public IP, EC2 private IP, and the SSH tunnel command strings), and the final confirmation "Destroy complete! Resources: 4 destroyed" confirms the complete and clean removal of all deployment-layer cloud resources.

```
PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\deployment> terraform destroy -auto-approve
data.aws_subnet.private: Reading...
data.aws_iam_instance_profile.ec2_deployment_profile: Reading...
data.aws_subnet.public: Reading...
data.aws_vpc.main: Reading...
data.aws_iam_instance_profile.ec2_deployment_profile: Read complete after 1s [id=AIPA2U3H6JJFUKMXTLFH]
data.aws_subnet.public: Read complete after 1s [id=subnet-06ab8b608fd0becd0]
data.aws_subnet.private: Read complete after 1s [id=subnet-099bb09528fbee1b8]
data.aws_vpc.main: Read complete after 2s [id=vpc-0962a0563e0d101bb]
aws_security_group.bastion: Refreshing state... [id=sg-03576654f8462f493]
aws_security_group.ec2_deployment: Refreshing state... [id=sg-0e57a3183a833138a]
aws_instance.bastion: Refreshing state... [id=i-0a4aaec4609b74f03]
aws_instance.deployment: Refreshing state... [id=i-058d0b5841d59f703]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  - destroy

Terraform will perform the following actions:

# aws_instance.bastion will be destroyed
- resource "aws_instance" "bastion" {
  - ami              = "ami-0e86e20dae9224db8" -> null
  - arn              = "arn:aws:ec2:us-east-1:731973044811:instance/i-0a4aaec4609b74f03" -> null
```



```

Plan: 0 to add, 0 to change, 4 to destroy.

Changes to Outputs:
- bastion_public_ip      = "54.160.27.80" -> null
- curl_test_command      = "curl http://localhost:11434/api/generate -d '{\"model\":\"25nplx-llama3b\", \"prompt\": \"\n\nHow do I check disk usage in Ubuntu?\", \"stream\": false}''" -> null
- ec2_private_ip         = "10.0.2.76" -> null
- ollama_tunnel_command   = "ssh -i 25nplx-keypair.pem -L 11434:10.0.2.76:11434 ubuntu@54.160.27.80" -> null
- openwebui_tunnel_command = "ssh -i 25nplx-keypair.pem -L 8080:10.0.2.76:8080 ubuntu@54.160.27.80" -> null
- ssh_to_bastion          = "ssh -i 25nplx-keypair.pem ubuntu@54.160.27.80" -> null
- ssh_to_llm_via_bastion  = "ssh -i 25nplx-keypair.pem -J ubuntu@54.160.27.80 ubuntu@10.0.2.76" -> null

aws_instance.bastion: Destroying... [id=i-0a4aaec4609b74f03]
aws_instance.bastion: Still destroying... [id=i-0a4aaec4609b74f03, 00m10s elapsed]
aws_instance.deployment: Still destroying... [id=i-058d0b5841d59f703, 00m10s elapsed]
aws_instance.bastion: Still destroying... [id=i-0a4aaec4609b74f03, 00m20s elapsed]
aws_instance.deployment: Still destroying... [id=i-058d0b5841d59f703, 00m20s elapsed]
aws_instance.bastion: Still destroying... [id=i-0a4aaec4609b74f03, 00m30s elapsed]
aws_instance.deployment: Still destroying... [id=i-058d0b5841d59f703, 00m30s elapsed]
aws_instance.bastion: Destruction complete after 31s
aws_instance.deployment: Still destroying... [id=i-058d0b5841d59f703, 00m40s elapsed]
aws_instance.deployment: Destruction complete after 41s
aws_security_group.ec2_deployment: Destroying... [id=sg-0e57a3183a833138a]
aws_security_group.ec2_deployment: Destruction complete after 1s
aws_security_group.bastion: Destroying... [id=sg-03576654f8462f493]
aws_security_group.bastion: Destruction complete after 1s

Destroy complete! Resources: 4 destroyed.

```

Fig. (41): Terminal output of terraform destroy -auto-approve for the deployment/ module, confirming the sequential destruction of all four deployment resources: both EC2 instances (bastion and LLM) and both security groups, with a total elapsed time of approximately 41 seconds for the LLM instance destruction. The final message confirms "Destroy complete! Resources: 4 destroyed."

As the final step of the project lifecycle, the foundational networking and identity infrastructure was decommissioned by navigating to the foundation/ module directory and executing terraform destroy -auto-approve. As shown in Fig. (42), Terraform systematically destroyed all 19 provisioned resources in reverse dependency order: IAM policy attachments and instance profiles were destroyed first, followed by route table associations, route tables, the private and public subnets, the VPC S3 endpoint, IAM roles, the NAT Gateway (which required approximately 42 seconds to fully release), the Internet Gateway, the Elastic IP, and finally the VPC itself. The complete destruction of all resources confirms that no AWS infrastructure components — and therefore no associated costs — persist beyond the conclusion of the project. The final message "Destroy complete! Resources: 19 destroyed" marks the successful and complete teardown of the entire cloud infrastructure deployed for this project.

```

PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\deployment> cd ..
PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files> cd foundation
PS D:\Queens_university\CISC_886_Cloud_Computing_W26\Project\files\Terraform_files\foundation> terraform destroy -auto-approve

aws_eip.nat: Refreshing state... [id=eipalloc-0ac9ef146039d80e7]
aws_iam_role.emr_ec2_role: Refreshing state... [id=25nplx-emr-ec2-role]
aws_iam_role.emr_service_role: Refreshing state... [id=25nplx-emr-service-role]
aws_vpc.main: Refreshing state... [id=vpc-0962a0563e0d101bb]
aws_iam_role.ec2_deployment_role: Refreshing state... [id=25nplx-ec2-deployment-role]
aws_iam_role_policy_attachment.emr_service_policy: Refreshing state... [id=25nplx-emr-service-role-20260430100947257500000001]
aws_iam_role_policy_attachment.emr_ec2_policy: Refreshing state... [id=25nplx-emr-ec2-role-20260430100947427000000002]
aws_iam_instance_profile.emr_ec2_profile: Refreshing state... [id=25nplx-emr-ec2-profile]
aws_iam_role_policy.ec2_s3_policy: Refreshing state... [id=25nplx-ec2-deployment-role:25nplx-ec2-s3-access]
aws_iam_instance_profile.ec2_deployment_profile: Refreshing state... [id=25nplx-ec2-deployment-profile]
aws_internet_gateway.igw: Refreshing state... [id=igw-07561dd035d3c93d2]
aws_subnet.private: Refreshing state... [id=subnet-099bb09528fbee1b8]
aws_subnet.public: Refreshing state... [id=subnet-06ab8b608fd0becd0]
aws_route_table.public: Refreshing state... [id=rtb-0e0c98ccf2ec8cb2a]
aws_nat_gateway.nat: Refreshing state... [id=nat-01d44224acd901dbe]
aws_route_table_association.public: Refreshing state... [id=rtbassoc-00a4bcfa57ad62dbc]
aws_route_table.private: Refreshing state... [id=rtb-030253de42e0ea36c]
aws_route_table_association.private: Refreshing state... [id=rtbassoc-00723023574eabc35]
aws_vpc_endpoint.s3: Refreshing state... [id=vpce-00871980237faf06a]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
- destroy

Terraform will perform the following actions:

# aws_eip.nat will be destroyed
- resource "aws_eip" "nat" {
  - allocation id      = "eipalloc-0ac9ef146039d80e7" -> null

```

```

aws_iam_instance_profile.ec2_deployment_profile: Destroying... [id=25nplx-ec2-deployment-profile]
aws_iam_role_policy_attachment.emr_ec2_policy: Destroying... [id=25nplx-emr-ec2-role-20260430100947427000000002]
aws_iam_role_policy_attachment.emr_service_policy: Destroying... [id=25nplx-emr-service-role-20260430100947257500000001]
aws_route_table_association.private: Destroying... [id=rtbassoc-00723023574eabc35]
aws_route_table_association.public: Destroying... [id=rtbassoc-00a4bcfa57ad62dbc]
aws_iam_instance_profile.emr_ec2_profile: Destroying... [id=25nplx-emr-ec2-profile]
aws_iam_role_policy.ec2_s3_policy: Destroying... [id=25nplx-ec2-deployment-role:25nplx-ec2-s3-access]
aws_vpc_endpoint.s3: Destroying... [id=vpce-00871980237faf06a]
aws_iam_role_policy.ec2_s3_policy: Destruction complete after 0s
aws_iam_role_policy_attachment.emr_ec2_policy: Destruction complete after 0s
aws_iam_role_policy_attachment.emr_service_policy: Destruction complete after 0s
aws_iam_role.emr_service_role: Destroying... [id=25nplx-emr-service-role]
aws_iam_instance_profile.emr_ec2_profile: Destruction complete after 1s
aws_iam_role.emr_ec2_role: Destroying... [id=25nplx-emr-ec2-role]
aws_iam_instance_profile.ec2_deployment_profile: Destruction complete after 1s
aws_iam_role.ec2_deployment_role: Destroying... [id=25nplx-ec2-deployment-role]
aws_route_table_association.public: Destruction complete after 1s
aws_route_table_association.private: Destruction complete after 1s
aws_subnet.private: Destroying... [id=subnet-099bb09528fbee1b8]
aws_iam_role.emr_service_role: Destruction complete after 0s
aws_iam_role.emr_ec2_role: Destruction complete after 0s
aws_iam_role.ec2_deployment_role: Destruction complete after 0s
aws_subnet.private: Destruction complete after 1s
aws_vpc_endpoint.s3: Destruction complete after 6s
aws_route_table.public: Destroying... [id=rtb-0e0c98ccf2ec8cb2a]
aws_route_table.private: Destroying... [id=rtb-030253de42e4bea36c]
aws_route_table.public: Destruction complete after 1s
aws_route_table.private: Destruction complete after 1s
aws_nat_gateway.nat: Destroying... [id=nat-01d44224acd901dbe]
aws_nat_gateway.nat: Still destroying... [id=nat-01d44224acd901dbe, 00m10s elapsed]
aws_nat_gateway.nat: Still destroying... [id=nat-01d44224acd901dbe, 00m20s elapsed]
aws_nat_gateway.nat: Still destroying... [id=nat-01d44224acd901dbe, 00m30s elapsed]
aws_nat_gateway.nat: Still destroying... [id=nat-01d44224acd901dbe, 00m40s elapsed]
aws_nat_gateway.nat: Destruction complete after 42s
aws_internet_gateway.igw: Destroying... [id=igw-07561dd035d3c93d2]
aws_eip.nat: Destroying... [id=eipalloc-0ac9ef146039d80e7]
aws_subnet.public: Destroying... [id=subnet-06ab8b608fd0becd0]
aws_internet_gateway.igw: Destruction complete after 1s
aws_subnet.public: Destruction complete after 1s
aws_vpc.main: Destroying... [id=vpc-0962a0563e0d101bb]
aws_eip.nat: Destruction complete after 1s
aws_vpc.main: Destruction complete after 1s

Destroy complete! Resources: 19 destroyed.

```

Fig. (42): Terminal output of terraform destroy -auto-approve for the foundation/ module, confirming the complete teardown of all 19 foundational AWS resources, including the VPC, public and private subnets, Internet Gateway, NAT Gateway, Elastic IP, route tables, IAM roles and instance profiles, and the S3 VPC endpoint. The NAT Gateway destruction required approximately 42 seconds, and the final message confirms "Destroy complete! Resources: 19 destroyed."

Cost summary table

Service	Usage	Estimated cost
EC2 t3.2xlarge (LLM server)	~0.5 hours	~\$0.08
EC2 t3.micro (bastion)	~0.5 hours	~\$0.01
EMR 3× m5.xlarge	~0.25 hours	~\$0.60
NAT Gateway	~3 GB (Docker + model pull)	~\$0.14
S3 storage	~1.5 GB	~\$0.04
Elastic IP	~0.5 hours	~\$0.01
Total		~\$0.88